

Uvod u Računalni vid



Sadržaj

- Što je Računalni vid?
- Prezentacija digitalne slike
- Sustavi boja
- Poduzorkovanje boje u odnosu na svjetlinu

Što je računalni vid?

Što je Računalni vid?

- Engl. **Computer Vision (CV)**
- Interdisciplinarno znanstveno područje u sklopu kojeg se pokušavaju razviti tehnike koje **pomažu računalima da "vide" i razumiju sadržaj digitalnih slika i videozapisa**
 - Ljudi te zadatke rješavaju vrlo jednostavno i brzo uče (čak i djeca)
- Računalni vid nije do kraja istražen
 - Ograničeno razumijevanje biološkog vida
 - Složenost vizualne percepcije u dinamičkom i neprestano promjenjivom fizičkom svijetu

Što uključuje Računalni vid?

- Uključuje metode za **prikupljanje, obradu, analizu i razumijevanje** digitalnih slika te **izdvajanje visoko-dimenzionalnih podataka**, kako bi se dobile nekakve numeričke ili simboličke informacije
- **Razumijevanje** – transformacija vizualnih slika u opis svijeta, s ciljem odgovarajućeg djelovanja
- Neki od konkretnih zadataka:
 - Prepoznavanje objekata
 - Praćenje objekata
 - 3D rekonstrukcija scene
 - Estimacija položaja (poze) objekta
 - ...

Primjena Računalnog vida

- Gdje se sve primjenjuje računalni vid?
 - Upravljanje robotima (strojevima)
 - Poljoprivreda
 - Videonadzor
 - Autonomna vožnja
 - Autonomna vozila → cilj je da u budućnosti vozila voze potpuno samostalno, bez potrebe za vozačem
 - Vozač će biti putnik kao i ostali suputnici
 - ...
- **Tehnike računalnog vida → kroz primjenu u naprednim sustavima pomoći vozaču pri vožnji**

Prezentacija digitalne slike

Digitalizacija slike

- Zašto je potrebno digitalizirati sliku?
- Što znači digitalizirati sliku?
- Digitalna slika sastoji se od piksela (elemenata slike), koji imaju svoj položaj u slici i svoju vrijednost svjetline/boje
- Digitalizacija slike sastoji se od dva postupka
 - Prostorno uzorkovanje
 - Prostor je kontinuiran, a prikazujemo ga s konačnim brojem diskretnih točaka (piksela)
 - Kvantizacija
 - Postoji beskonačno mnogo nijansi svjetline/boje u prostoru (kontinuiranog opsega), a zapisujemo ih s nekim konačnim brojem bita, tj. konačnim brojem svjetlina/boja

Digitalizacija slike – prostorno uzorkovanje

- **Prostorno uzorkovanje**

- određuje prostornu rezoluciju slike
- broj senzora ograničava broj uzoraka po jedinici duljine

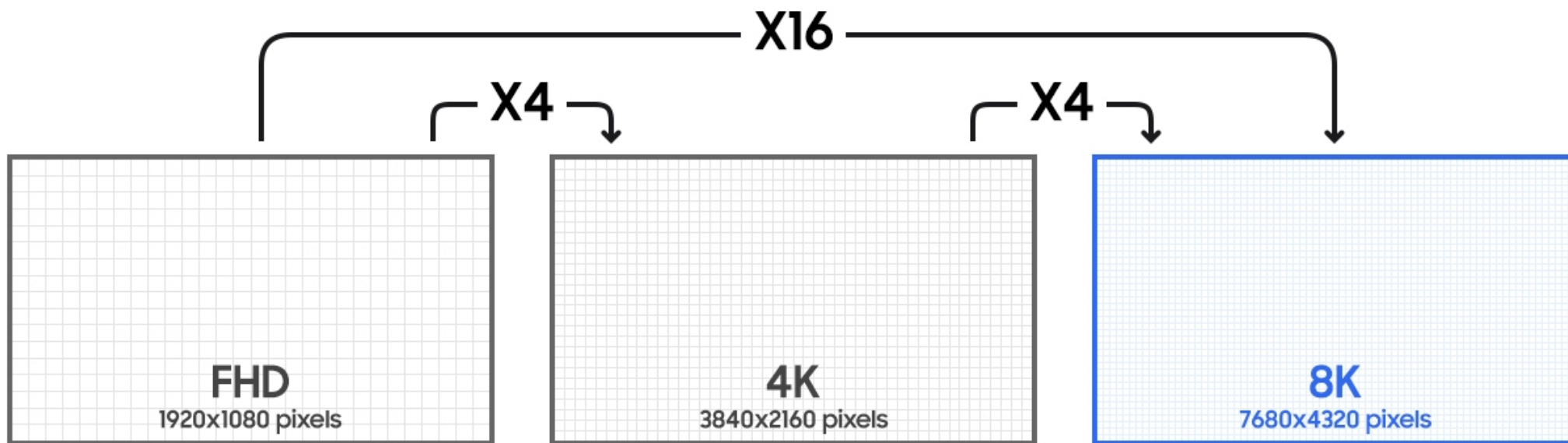
- Npr. neke često korištene prostorne rezolucije (širina x visina)

- CIF – 352 x 288 piskela
- VGA – 640 x 480 piksela
- HD – 1280 x 720 piksela
- Full HD – 1920 x 1080 piksela
- Ultra HD (UHD) ili 4K – 3840 x 2160 piksela
- 8K – 7680 x 4320 piksela

Digitalizacija slike – prostorno uzorkovanje

- **Prostorno uzorkovanje**

- određuje prostornu rezoluciju slike
- broj senzora ograničava broj uzoraka po jedinici duljine



Digitalizacija slike – učinci prostornog uzorkovanja



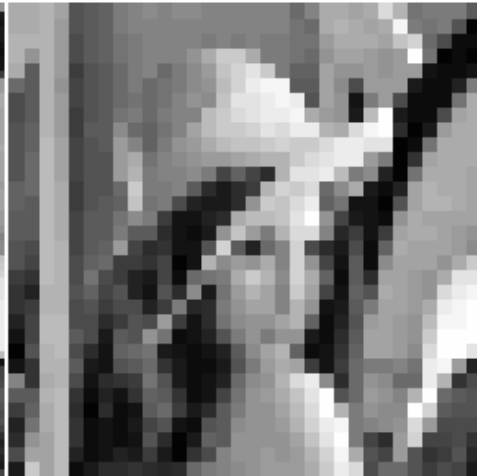
256 x 256



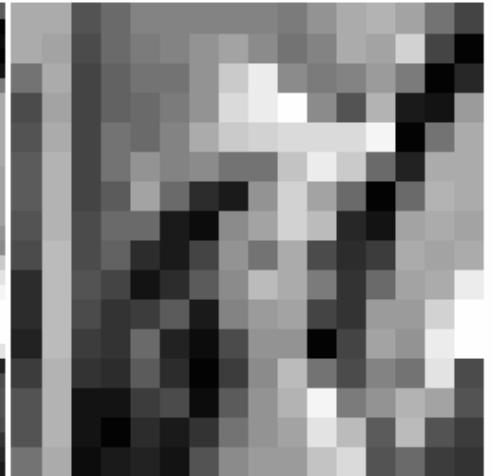
128 x 128



64 x 64



32 x 32



16 x 16

Digitalizacija slike – kvantizacija

- **Kvantizacija**

- određuje s koliko je različitih razina (amplituda) moguće zapisati svjetlinu ili određenu komponentu boje za pojedini piksel slike
 - broj kvantizacijskih razina ovisi o zahtjevima primjene slike (nekad je dovoljno manje, nekad je potrebno više razina)
-
- Koliko različitih razina svjetline piksela je moguće zapisati u digitalnom obliku, ako se za njihov zapis koristi:
 - 1 bit ?
 - 2 bita ?
 - 8 bita ?

Digitalizacija slike – kvantizacija

- Ako se za zapis vrijednosti svjetline/boje pojedinog piksela slike koristi k bita, onda:
 - Govorimo o **k-bitnoj slici**
 - **Moguće je zapisati 2^k različitih razina (vrijednosti) piksela**
 - Broj k se još naziva i **dubina boje**
- Primjeri
 - $k=1 \rightarrow$ 1-bitna slika (često se naziva i binarna slika – dvije moguće razine: 0 i 1)
 - $k=2 \rightarrow$ 2-bitna slika (ima četiri (2^2) moguće razine: 00,01,10,11)
 - $k=8 \rightarrow$ 8-bitna slika (ima $2^8=256$ mogućih razina: 00000000,...,11111111)

Digitalizacija slike – učinci kvantizacije

k=8 (256 razina)



k=7 (128 razina)



k=6 (64 razine)



k=5 (32 razine)



k=4 (16 razina)



k=3 (8 razina)

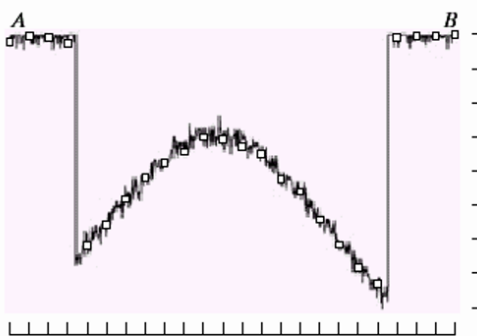
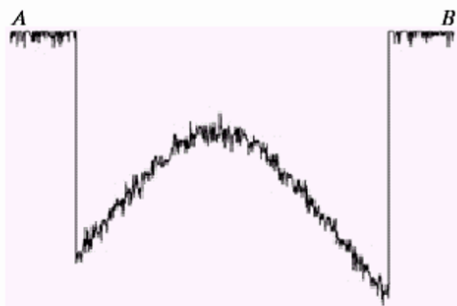
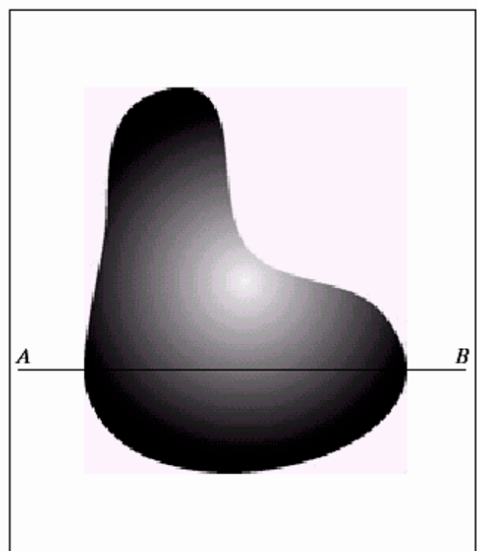


k=2 (4 razine)

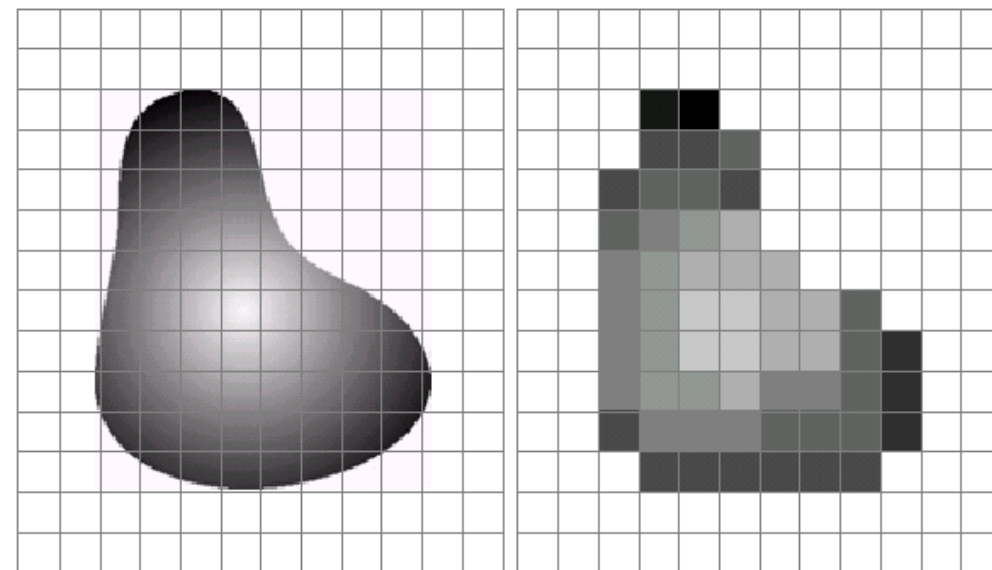
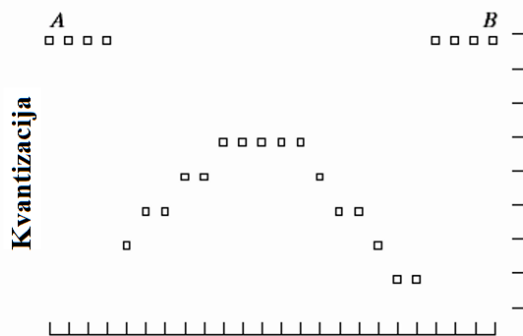


k=1 (2 razine)

Digitalizacija slike – uzorkovanje i kvantizacija



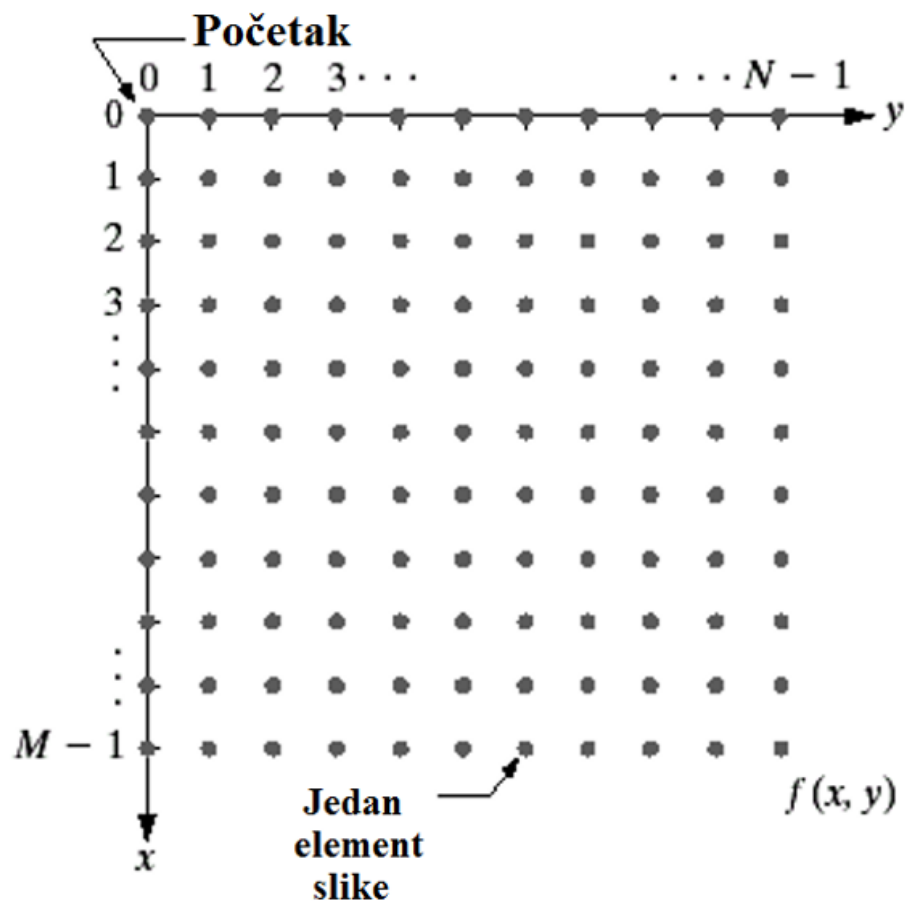
Uzorkovanje



- Slika nakon uzorkovanja i kvantizacije koisteći 3 bita za zapis pojedinog piksela
- Broj senzora postavlja granice na uzorkovanje u oba smjera
- Broj bita za zapis vrijednosti pojedinog piksela određuje maksimalni broj mogućih nijansi

Predstavljanje digitalne slike

- Slika koja ima prostornu rezoluciju općenito $N \times M$ piksela



$$f(x, y) = \begin{bmatrix} f(0, 0) & f(0, 1) & \cdots & f(0, N-1) \\ f(1, 0) & f(1, 1) & \cdots & f(1, N-1) \\ \vdots & \vdots & & \vdots \\ f(M-1, 0) & f(M-1, 1) & \cdots & f(M-1, N-1) \end{bmatrix}$$

- Rezultat uzorkovanja i kvantizacije je matrica realnih brojeva (ima M redaka i N stupaca)
- **Digitalna slika je dvodimenzionalno (2D) polje**

P i t a n j a ?

Sustavi boja

Boja u slikama – DA/NE?

- Je li boja uvijek neophodna u slikama?
- Slike bez boje
 - Zahtjevaju manje memorije pri pohrani i prijenosu
 - Otporne su na razlike u prikazu boje na različitim zaslonima
- Ipak...
 - Boja u velikom broju slučajeva donosi informaciju koja je od ključnog značaja

Jednobojna (monokromatska) slika

- Ako se slika prikazuje samo u nijansama jedne boje, ona je predstavljena jednom 2D matricom
- Tada se radi o jednobojnoj (monokromatskoj) slici
- Npr. samo u nijansama sive boje (*grayscale* slika)

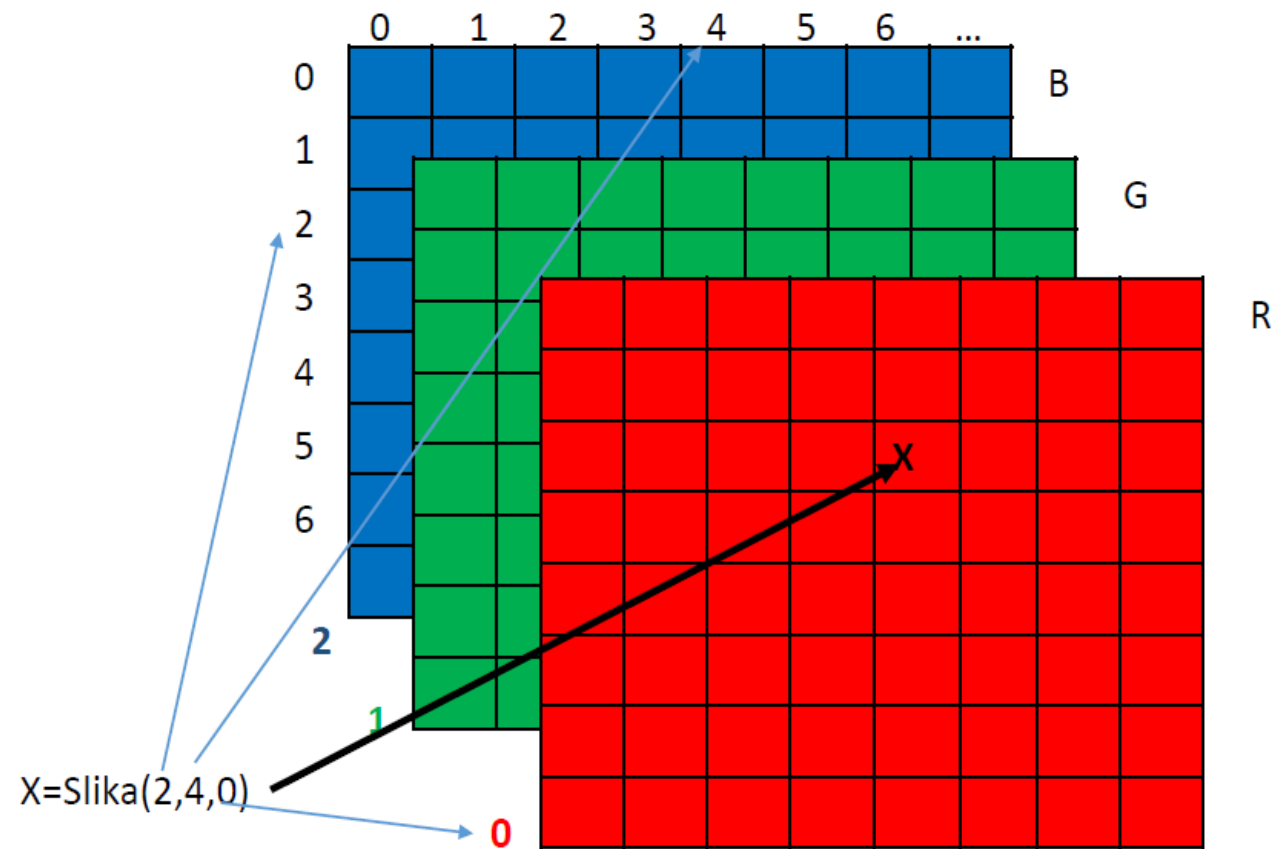


Slika u boji (kromatska slika)

- Za kromatsku sliku potrebne su 3 komponente boje, tj. 3 dvodimenzionalne (2D) matrice
 - Svaka je komponenta za pojedini piksel predstavljena s k bita → ukupno $3 \times k$ bita
- Slika u boji je 3-dimenzionalno polje jer ima 3 dimenzije
 - širina
 - visina
 - „dubina” (ona iznosi 3 jer postoje 3 komponente boje)
- Postoje različiti sustavi boja → upoznat ćemo se s njima u nastavku...

Sustavi boja - RGB

- Primari RGB: crvena, plava i zelena
- Koriste se za dobivanje boja aditivnim miješanjem – to je aditivni model boja
- Koristi se u HDTV (High Definition TV) – televizija visoke kvalitete



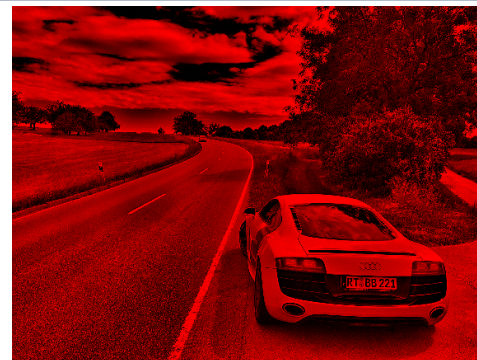
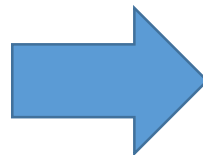
Sustavi boja - RGB

- S RGB primarima predstavlja se “količina” **crvene**, **zelene** i **plave** svjetlosti, što se može izraziti postotkom u odnosu na zasićenu boju
- Ako se npr. u digitalnoj formi za svaki **primar** koristi 8 bita, tj. 256 razina boje, **to daje ukupno $256 * 256 * 256 = 16\,777\,216$ različitih kombinacija primara, tj. različitih boja u slici**
- Primjeri:
 - (100%, 0%, 0%) (255, 0, 0)
 - (100%, 50%, 100%) (255, 127, 255)
 - (100%, 100%, 0%) (255, 255, 0)
 - (0%, 0%, 0%) (0, 0, 0)
 - (100%, 100%, 100%) (255, 255, 255)

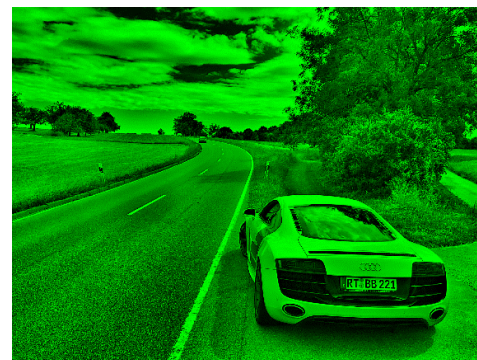


Sustavi boja - RGB

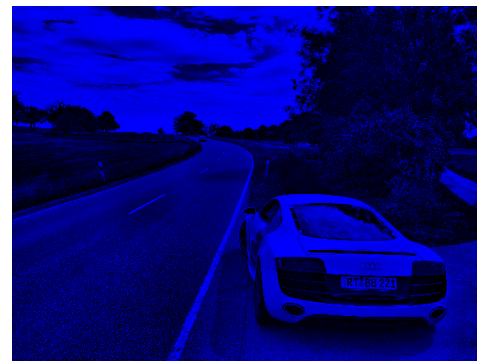
Originalna slika



R



G



B

Sustavi boja - YUV

- **Fizikalne činjenice**

- Ljudski vizualni sustav osjetljiviji je na promjenu luminancije (svjetline) nego na promjenu boje
- „Mala” promjena svjetline znatnije mijenja sadržaj/značenje slike, nego isto taliko „mala” promjena boje („mala” = za neki određeni mali iznos)

- YUV model boja → koristi se za **digitalni video**

- YUV model pogodniji za obradu i kompresiju slike i videa → vidjet ćemo na brojnim primjerima zašto...

Sustavi boja - YUV

- YUV komponente za pojedini piksel dobiju se linearnom transformacijom iz **RGB** komponentenata
 - Tri komponente $\rightarrow$ Y, U i V

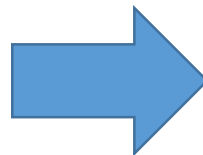
$$Y = 0,299R + 0,587G + 0,114B$$

$$U = 0,493(B - Y) = -0,15R - 0,29G + 0,44B$$

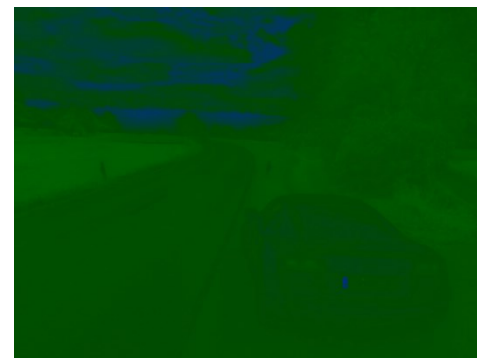
$$V = 0,877(R - Y) = 0,62R - 0,52G - 0,10B$$

Sustavi boja - YUV

Originalna slika



Y

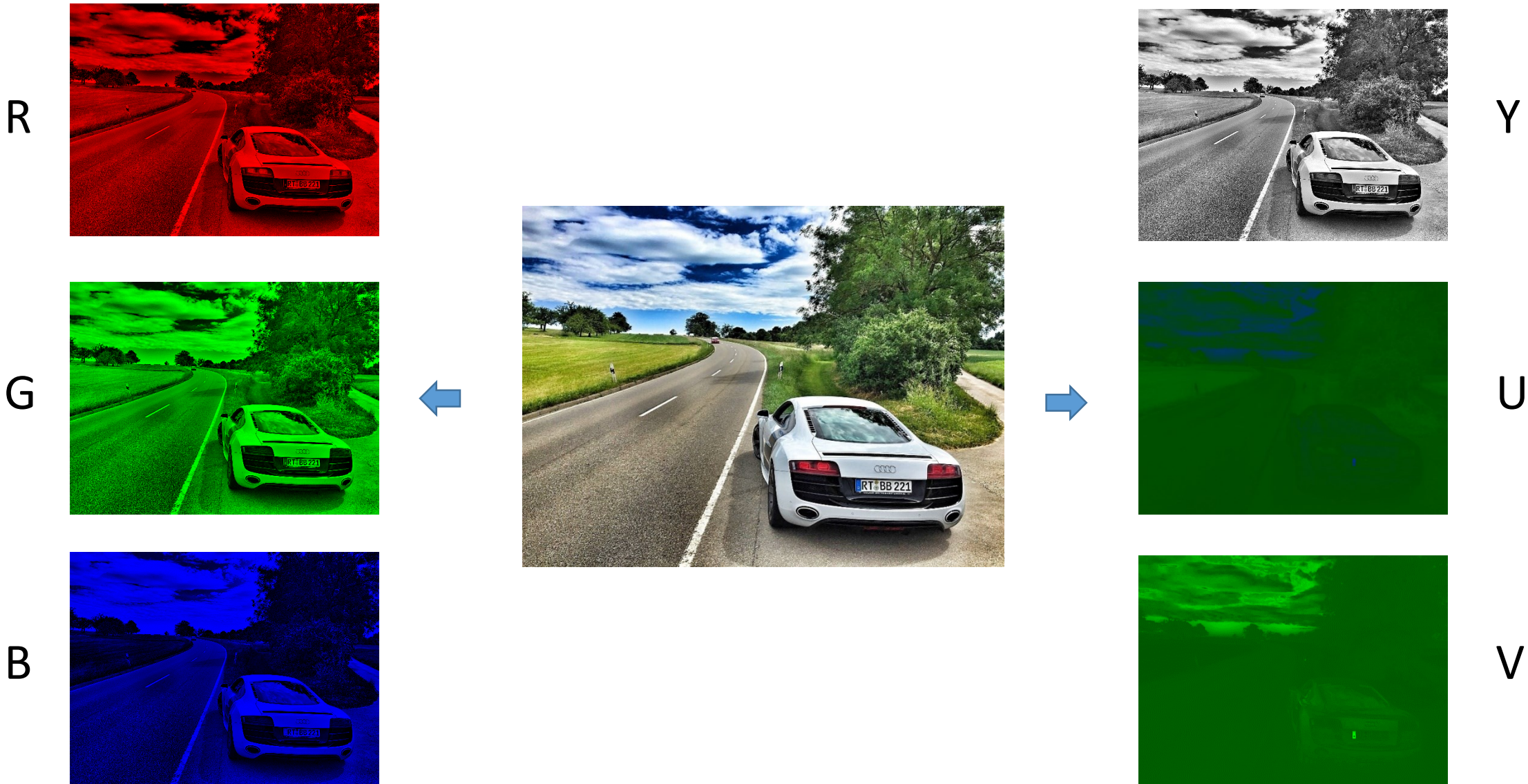


U



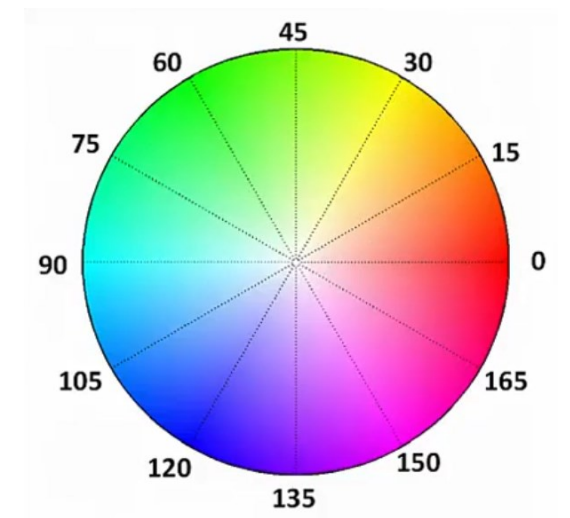
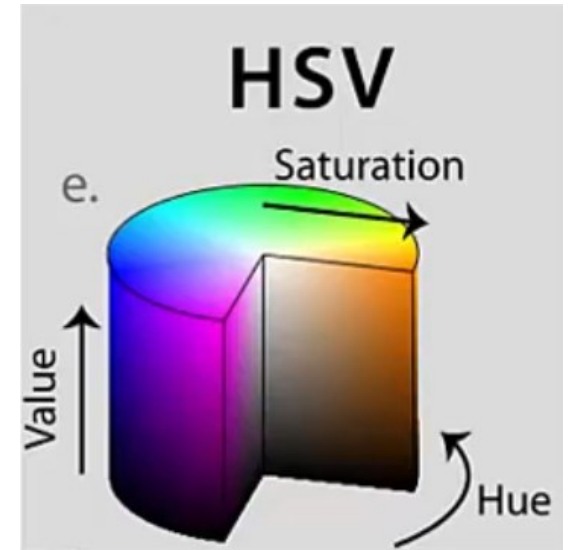
V

Sustavi boja – RGB vs. YUV



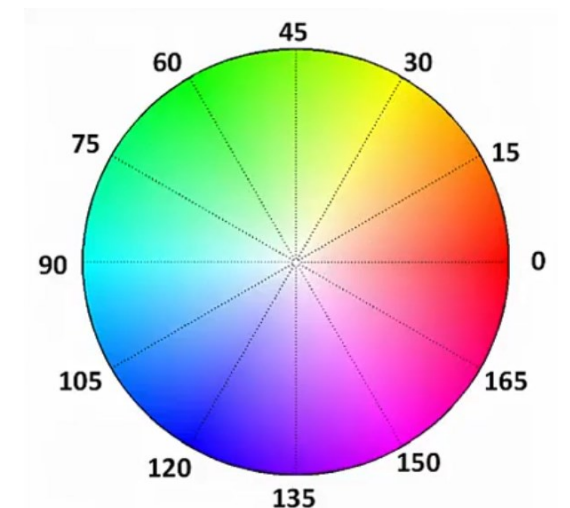
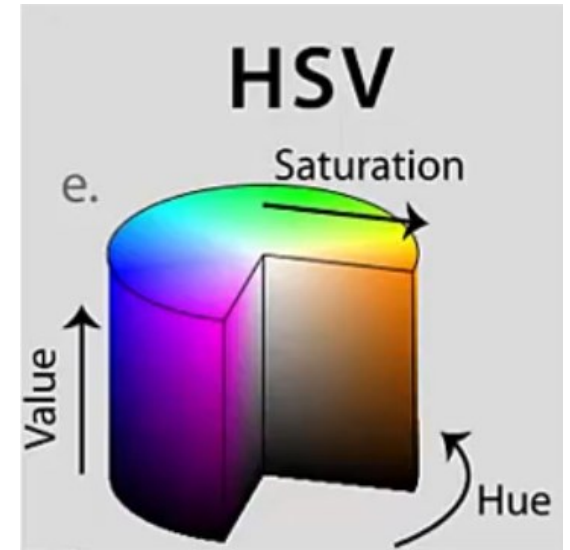
Sustavi boja - HSV

- Tri komponente za svaki piksel (HSV)
 - Hue (boja)
 - Saturation (zasićenje)
 - Value/Brightness (vrijednost/osvjetljenje)
- Pogodan u određenim aplikacijama zasnovanim na računalnom vidu, npr.
 - Filtriranje objekata po boji
 - Praćenje objekata određene boje



Sustavi boja - HSV

- Filtriranje prema *Hue* komponenti (boja)
 - Hue → opseg boja ide od 0 do 180
 - Filtri za pojedinu boju:
 - Crvena – 165 do 15
 - Zelena – 45 do 75
 - Plava – 90 do 120
- Filtriranje prema *Saturation* i *Value/Brightness*
 - Obično se postavlja opseg filtra od 50 do 255 za zasićenje i osvjetljenje jer time zadržavamo zapravo stvarne boje
 - *Saturation* od 0 do 50 je vrlo blizu bijele boje (svjetlije boje)
 - *Value* od 0 do 60 je vrlo blizu crne boje (tamnije boje)



Učitavanje i prikaz slike u OpenCV

- OpenCV – Open Source Computer Vision
- Pisan u C++
- Postoje verzije za Python, Matlab, Java

```
import cv2

# Load an image using 'imread' specifying the path to image
input = cv2.imread('./images/car_and_road.jpg')

# Our file 'car_and_road.jpg' is now loaded and stored in python
# as a variable we named 'input'

# To display our input variable, we use 'imshow'
# The first parameter will be title shown on image window
# The second parameter is the image variable
cv2.imshow('Auto i cesta - originalna slika', input)

# 'waitKey' allows us to input information when a image window is open
# By leaving it blank it just waits for anykey to be pressed before
# continuing. By placing numbers (except 0), we can specify a delay for
cv2.waitKey()

# This closes all open windows
# Failure to place this will cause your program to hang
cv2.destroyAllWindows()

print input.shape
# The 2D dimensions are 292 pixels height and 392 pixels wide.
# The '3L' means that there are 3 other components (RGB) that make up this image.

# Let's print each dimension of the image

print 'Height of Image:', int(input.shape[0]), 'pixels'
print 'Width of Image: ', int(input.shape[1]), 'pixels'

(292L, 392L, 3L)
Height of Image: 292 pixels
Width of Image: 392 pixels
```

Pretvaranje slike u monokromatsku, Prebacivanje iz BGR u HSV sustav boja

```
import cv2
import numpy as np

image = cv2.imread('./images/car_and_road.jpg')
```

```
# BGR Values for the pixel at location (10,50)
# OpenCV stores the image in BGR
B, G, R = image[10, 50]
print B, G, R
print image.shape
```

```
252 197 170
(292L, 392L, 3L)
```

```
gray_img = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
print gray_img.shape
print gray_img[10, 50]
```

```
(292L, 392L)
195
```

```
gray_img[10, 50]
```

```
195
```

```
#H: 0 - 180, S: 0 - 255, V: 0 - 255
```

```
image = cv2.imread('./images/car_and_road.jpg')

hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

cv2.imshow('HSV image', hsv_image)
cv2.imshow('Hue channel', hsv_image[:, :, 0])
cv2.imshow('Saturation channel', hsv_image[:, :, 1])
cv2.imshow('Value channel', hsv_image[:, :, 2])

cv2.waitKey()
cv2.destroyAllWindows()
```

- **NAPOMENA** → OpenCV pohranjuje sliku kao **BGR**, a ne kao **RGB** → paziti !!!

Izdvajanje R, G i B kanala

Prikaz kanala u odgovarajućoj boji

```
image = cv2.imread('./images/car_and_road.jpg')

# OpenCV's 'split' function splits the image into each color index
B, G, R = cv2.split(image)

print B.shape
cv2.imshow("Red", R)
cv2.imshow("Green", G)
cv2.imshow("Blue", B)
cv2.waitKey(0)
cv2.destroyAllWindows()

# Let's re-make the original image,
merged = cv2.merge([B, G, R])
cv2.imshow("Merged", merged)

# Let's amplify the blue color
merged = cv2.merge([B+100, G, R])
cv2.imshow("Merged with Blue Amplified", merged)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
import cv2
import numpy as np

B, G, R = cv2.split(image)

# Let's create a matrix of zeros
# with dimensions of the image h x w
zeros = np.zeros(image.shape[:2], dtype = "uint8")

cv2.imshow("Red", cv2.merge([zeros, zeros, R]))
cv2.imshow("Green", cv2.merge([zeros, G, zeros]))
cv2.imshow("Blue", cv2.merge([B, zeros, zeros]))

cv2.waitKey(0)
cv2.destroyAllWindows()
```

P i t a n j a ?

Poduzorkovanje boje u odnosu na svjetlinu

Memorijski zahtjevi videa, zahtjevi pri prijenosu videa

- Razmotrimo sljedeće činjenice...
 - Većina suvremenih uređaja podržava FullHD (1920 x 1080 piksela), UHD (3840 x 2160 piksela) ili 8K (7680 x 4320 piksela)
 - Kada bi htjeli pohraniti FullHD, UDH ili 8K video u boji od 60 minuta koji izmjenjuje 30 okvira po sekundi te je svaki piksel **zapisan s 24 bita**, to bi zauzimalo

veličina videa (FullHD) = $1920 * 1080 * 24 \text{ bita} * 30 \text{ fps} * 60 \text{ s} * 60 \text{ min} \approx \text{670 GB}$

veličina videa (UHD) = $3840 * 2160 * 24 \text{ bita} * 30 \text{ fps} * 60 \text{ s} * 60 \text{ min} \approx \text{2.68 TB}$

veličina videa (8K) = $7680 * 4320 * 24 \text{ bita} * 30 \text{ fps} * 60 \text{ s} * 60 \text{ min} \approx \text{10.72 TB}$

- **Jako velika količina podataka, pa čak i za moderne diskove od nekoliko terabajta i moderne prijenosne sustave**

Memorijski zahtjevi videa, zahtjevi pri prijenosu videa

- Podsjetimo se...
 - Fizikalne činjenice
 - Ljudski vizualni sustav osjetljiviji je na promjenu luminancije (svjetline) nego na promjenu boje
 - „Mala” promjena svjetline znatnije mijenja sadržaj/značenje slike, nego isto taliko „mala” promjena boje („mala” = za neki određeni mali iznos)
- Zašto to ne iskoristiti ?
- Ali kako...?
 - Poduzorkovanje boje u odnosu na svjetlinu/luminaciju
 - Što to znači, kako se radi?

Poduzorkovanje boje u odnosu na svjetlinu/luminaciju

- Pri digitalizaciji videa često se koristi YUV signal (sustav boja) → Zašto?
 - Imamo odvojenu svjetlinu (Y) od komponenata boje (U, V)
- Ako se koristi YUV sustav boja, moguće je iskoristiti manju osjetljivost ljudskog vizualnog sustava na promjenu boje u odnosu na promjenu svjetline
 - Ideja je → poduzorkovati komponente boje
 - Što to znači?
- Poduzorkovanje (engl. *subsampling*) boje
 - odnos broja uzoraka luminantne komponente i broja uzoraka krominantnih komponenata kod uzorkovanja označava se kao Y : C1 : C2

Poduzorkovanje boje u odnosu na svjetlinu/luminaciju

- Konačno - Zašto se radi poduzorkovanje boje?
 - Ljudsko oko osjetljivije je na luminantnu komponentu nego na krominantnu komponentu
 - Poduzorkovanjem krominantnog signala veću razlučivost pridodajemo luminantnoj nego krominantnoj komponenti
 - Na optimalnim udaljenostima gledanja ne možemo opaziti smanjenu kvalitetu slike nastalu poduzorkovanjem krominantne komponente

Poduzorkovanje boje u odnosu na svjetlinu/luminaciju



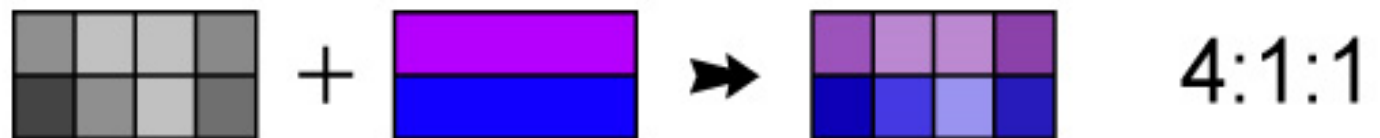
Izračunajmo uštedu zbog poduzorkovanja boje...



Ušteda 33 %

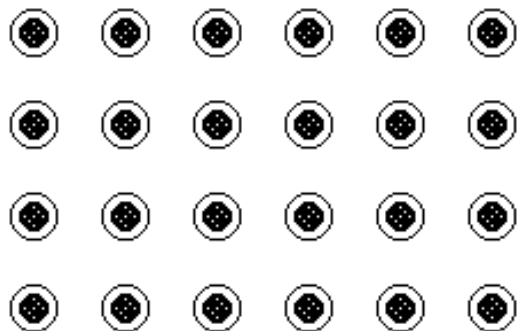


Ušteda 50 %

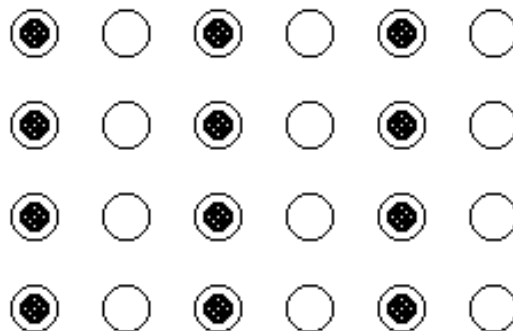


Ušteda 50 %

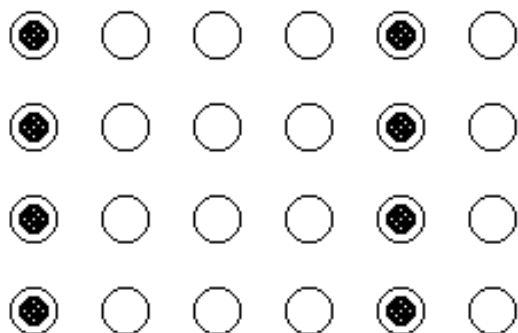
Poduzorkovanje boje u odnosu na svjetlinu/luminaciju



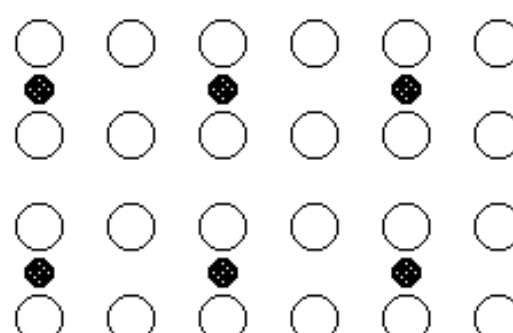
4:4:4



4:2:2



4:1:1

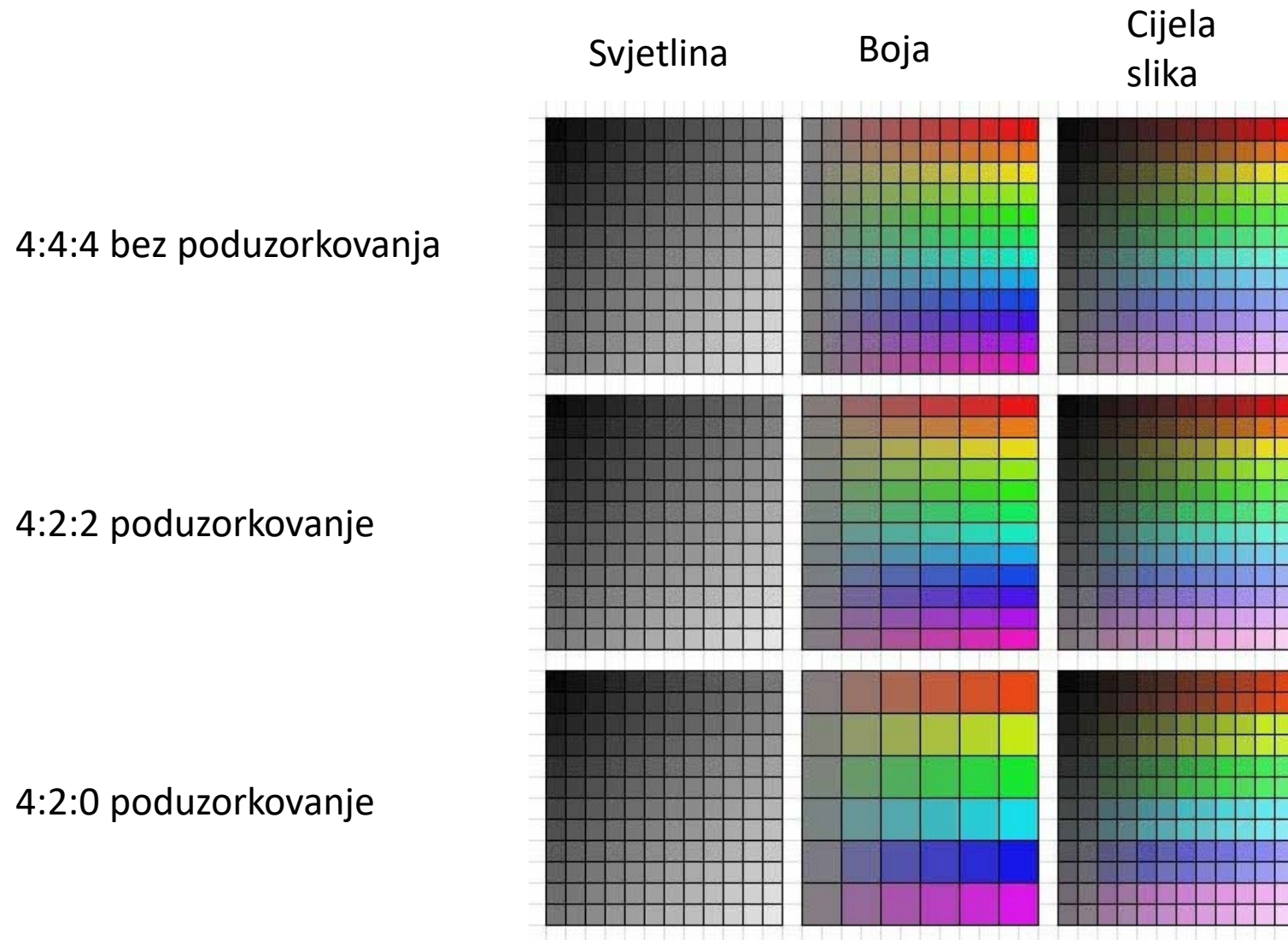


4:2:0

○ Píksel samo s Y komponentom

● Píksel s Y, U i V komponentom

Poduzorkovanje boje u odnosu na svjetlinu/luminaciju



Poduzorkovanje boje u odnosu na svjetlinu/luminaciju

- Primjer – učinak poduzorkovanja boje
- Gornji red slika prikazuje kompletnu sliku, a donji red prikazuje rezoluciju krominantnih dijelova



4:1:1



4:2:0



4:2:2



4:4:4



Moguće je primijetiti da je razlika u kvaliteti između slika gotovo pa neprimjetna – tek je na granicama između različitih boja kod slika s većim poduzorkovanjem krominantnih komponenata moguće primijetiti manje degradacije

Poduzorkovanje boje u odnosu na svjetlinu/luminaciju

- 4:4:4 – nema poduzorkovanja; sve tri YUV komponente imaju istu stopu uzorkovanja
- 4:2:2 – rezolucija krominantnih komponentenata horizontalno se dvostruko smanjuje, čime se štedi 33.3% prostora u odnosu na okvir u kojem boja nije poduzorkovana
- 4:2:0 – rezolucija krominantnih komponentenata horizontalno i vertikalno se dvostruko smanjuje; ušteda prostora je 50 %
- 4:1:1 – rezolucija krominantnih komponentenata horizontalno se četverostruko smanjuje; štednja prostora u odnosu na nekomprimirani okvir je 50 %

Poduzorkovanje boje u odnosu na svjetlinu/luminaciju

- Originalni FullHD (1920x1080) video od 60 minuta koji izmjenjuje 30 okvira po sekundi te je svaki element slike zapisan s 24 bita

veličina originalnog videa (FullHD) = $1920 * 1080 * 24 \text{ bita} * 30 \text{ fps} * 60 \text{ s} * 60 \text{ min} \approx \text{670 GB}$

- Poduzorkovani signal 4:2:2
 - 1920 uzoraka po liniji za luminaciju
 - 4:2:2 shema poduzorkovanja
 - uzorci za boju uzimaju se za svaki drugi element slike u svakoj liniji
 - 960 uzoraka po liniji za svaku krominantnu komponentu
 - uz 8 bita po uzorku ovakvo uzorkovanje daje
 - $1920 + 960 + 960 = 3840$ elemenata slike/liniji
 - za zapis takvog videa potrebno je:

veličina poduzorkovanog videa (FullHD) = $(1920+960+960) * 1080 * 8 \text{ bita} * 30 \text{ fps} * 60 \text{ s} * 60 \text{ min} \approx \text{448 GB}$

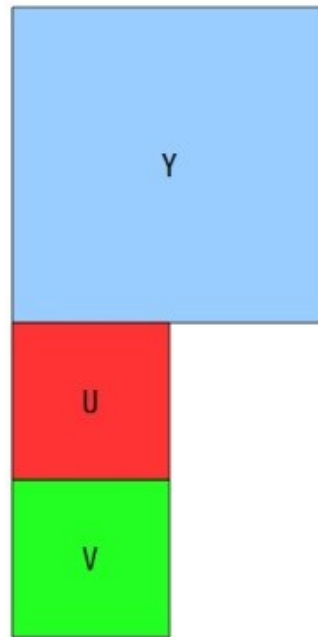
- ZAKLJUČAK → i dalje je video prevelik → potrebna kompresija, ali mi se tim dijelom obrade slike i videa ne bavimo...

Poduzorkovanje boje u odnosu na svjetlinu/luminaciju

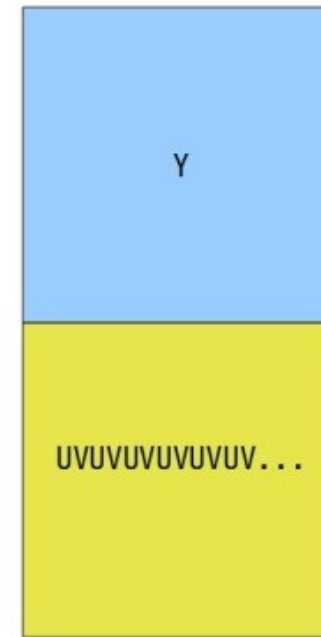
- Načini zapisa poduzorkovanih YUV signala slike...

- Planar
- Semi Planar
- Interleaved

Image Format(YUV)
planar, semi-planar, inter



Planar(420)



Semi Planar(422)



Interleaved

Poduzorkovanje boje u odnosu na svjetlinu/luminaciju

4:2:0 I420 p

Y1	Y2	Y3	Y4	Y5	Y6
Y7	Y8	Y9	Y10	Y11	Y12
Y13	Y14	Y15	Y16	Y17	Y18
Y19	Y20	Y21	Y22	Y23	Y24
U1	U2	U3	U4	U5	U6
V1	V2	V3	V4	V5	V6

4:2:0 NV12 sp

Y1	Y2	Y3	Y4	Y5	Y6
Y7	Y8	Y9	Y10	Y11	Y12
Y13	Y14	Y15	Y16	Y17	Y18
Y19	Y20	Y21	Y22	Y23	Y24
U1	V1	U2	V2	U3	V3
U4	V4	U5	V5	U6	V6

4:2:0 YV12 p

Y1	Y2	Y3	Y4	Y5	Y6
Y7	Y8	Y9	Y10	Y11	Y12
Y13	Y14	Y15	Y16	Y17	Y18
Y19	Y20	Y21	Y22	Y23	Y24
V1	V2	V3	V4	V5	V6
U1	U2	U3	U4	U5	U6

4:2:0 NV21 sp

Y1	Y2	Y3	Y4	Y5	Y6
Y7	Y8	Y9	Y10	Y11	Y12
Y13	Y14	Y15	Y16	Y17	Y18
Y19	Y20	Y21	Y22	Y23	Y24
V1	U1	V2	U2	V3	U3
V4	U4	V5	U5	V6	U6

Poduzorkovanje boje u odnosu na svjetlinu/luminaciju

Jedan okvir YUV420



Pozicija u nizu bajta



P i t a n j a ?

Uvod u Autonomnu vožnju



Sadržaj

- Autonomna vožnja i ADAS
- Senzori u ADAS
- Kamere za ADAS

Autonomna vožnja i ADAS

Zašto autonomna vožnja? Zašto Računalni vid?

- Tko u današnjem svijetu upravlja vozilima na prometnicama?
- Kojim osjetilom čovjek opaža najveću količinu informacije iz okoline?
 - 80-90% svih živčanih stanica ljudskog mozga uključeno u opažanje onoga što ljudsko oko gleda
- Zašto autonomna vožnja i zašto računalni vid?
 - Preko 70% nezgoda uzrokuje isključivo ljudski faktor
 - U preko 90% nezgoda uključen je ljudski faktor (gubitak kontrole nad vozilom, kasna reakcija zbog nepažnje ili umora, pogrešna reakcija,...)
 - Sustavi pomoći vozaču, zasnovani većinom na računalnom vidu, povećavaju sigurnost, efikasnost prijevoza, smanjuje troškove liječenja i popravka vozila, smanjuje zagađenje,...
 - [Pogledajmo video...](#)

Što su napredni sustavi pomoći vozaču?

- Napredni sustavi za pomoć vozaču u vožnji → **Advanced Driver Assistance Systems (ADAS)**
- **ADAS**
 - Elektronički sustav u vozilu koji pomaže vozaču u svakodnevnoj vožnji
 - Razvija se kako bi povećao sigurnost vožnje i općenito sigurnost prometa
 - Smanjuje utjecaj ljudske pogreške na način da alarmira vozača o opasnim situacijama ili preuzme kontrolu nad vozilom
 - Osnova je sustava za autonomnu vožnju
 - Sastoji se od naprednih senzora, snažne računalne podrške i aktuatora

Što su razine autonomije vožnje?

SAE razina	Naziv razine	Definicija	Skretanje i ubrzanje/ usporavanje	Praćenje voznog okruženja	Preuzimanje kontrole u slučaju potrebe	Modovi vožnje
Vozač prati vozno kruženje						
0	Nema automatizacije	Vozač u potpunosti kontrolira sve aspekte zadatka vožnje, čak i ako je vožnja unaprijedena sustavima upozorenja.	Vozač	Vozač	Vozač	-
1	Pomoć vozaču	Izvršavanje pojedinih sustava za pomoć vozaču: okretanje upravljača ili upravljanje s ubrzavanjem i usporavanjem u ovisnosti o okruženju, dok vozač izvodi sve ostale zadatke vožnje.	Vozač i sustav	Vozač	Vozač	Neki modovi vožnje
2	Djelomična automatizacija	Izvršavanje više sustava za pomoć vozaču u isto vrijeme: okretanje upravljača i upravljanje s ubrzavanjem i usporavanjem u ovisnosti o okruženju, dok vozač izvodi sve ostale zadatke vožnje.	Sustav	Vozač	Vozač	Neki modovi vožnje
Automatizirani sustav vožnje prati vozno okruženje						
3	Uvjetna automatizacija	Automatizacija zadatka vožnje u određenoj situaciji kontrolirajući sve aspekte vožnje, pri čemu vozač mora moći preuzeti upravljanje u slučaju potrebe.	Sustav	Sustav	Vozač	Neki modovi vožnje
4	Visoka automatizacija	Automatizacija zadatka vožnje u određenoj situaciji kontrolirajući sve aspekte vožnje i nastavak kontrole čak i ako vozač nije u stanju preuzeti upravljanje.	Sustav	Sustav	Sustav	Neki modovi vožnje
5	Potpuna automatizacija	Potpuna kontrola svih zadataka vožnje u svim situacijama koje bi inače mogao kontrolirati i vozač.	Sustav	Sustav	Sustav	Svi modovi vožnje

Što su razine autonomije vožnje?

- Trenutna razina autonomije vozila
 - Razina 1-2 – gotovo svi proizvođači u serijskoj proizvodnji
 - Razina 3 - neki proizvođači u serijskoj proizvodnji
 - Razina 4 - testni primjerci (Google, Audi, Tesla ...)
 - Razina 5 – još u razvoju
- Potrebno je zadovoljiti visoke sigurnosne zahtjeve
 - ISO 26262 “Road vehicles – Functional safety”
- Regulatora za autonomnu vožnju u razvoju
- [Razine autonomije – gdje smo?](#)

Senzori u ADAS

Senzori u ADAS

- Dinamički senzori
 - Brzine, akceleracije, promjene putanje, kuta upravljanja, tlaka ...
- Ultrazvučni senzori
 - Pomoć kod parkiranja
- RADAR (Radio Detection and Ranging)
 - Mjerenje udaljenosti i brzine (sustav zaštite i prevencije od sudara, pomoć kod promjene vozne trake)
- LIDAR (Light Detection And Ranging)
 - Mjerenje udaljenosti, detekcija objekata, mjerenje vidljivosti, procjena brzine
- **Video kamere**
 - **Nadzor unutrašnjosti vozila i njegove okoline**

Senzori u ADAS

UBER ATC

Top mounted lidar units provide a 360° 3-dimensional scan of the environment.

Forward facing camera array focus both close and far field, watching for breaking vehicles, crossing pedestrians, traffic lights, and signage.

Side and rear facing stereo camera pairs work in collaboration to construct a continuous view of the vehicle's surroundings

Roof and trunk mounted antennae provide GPS positioning and wireless data capabilities.

360° radar coverage

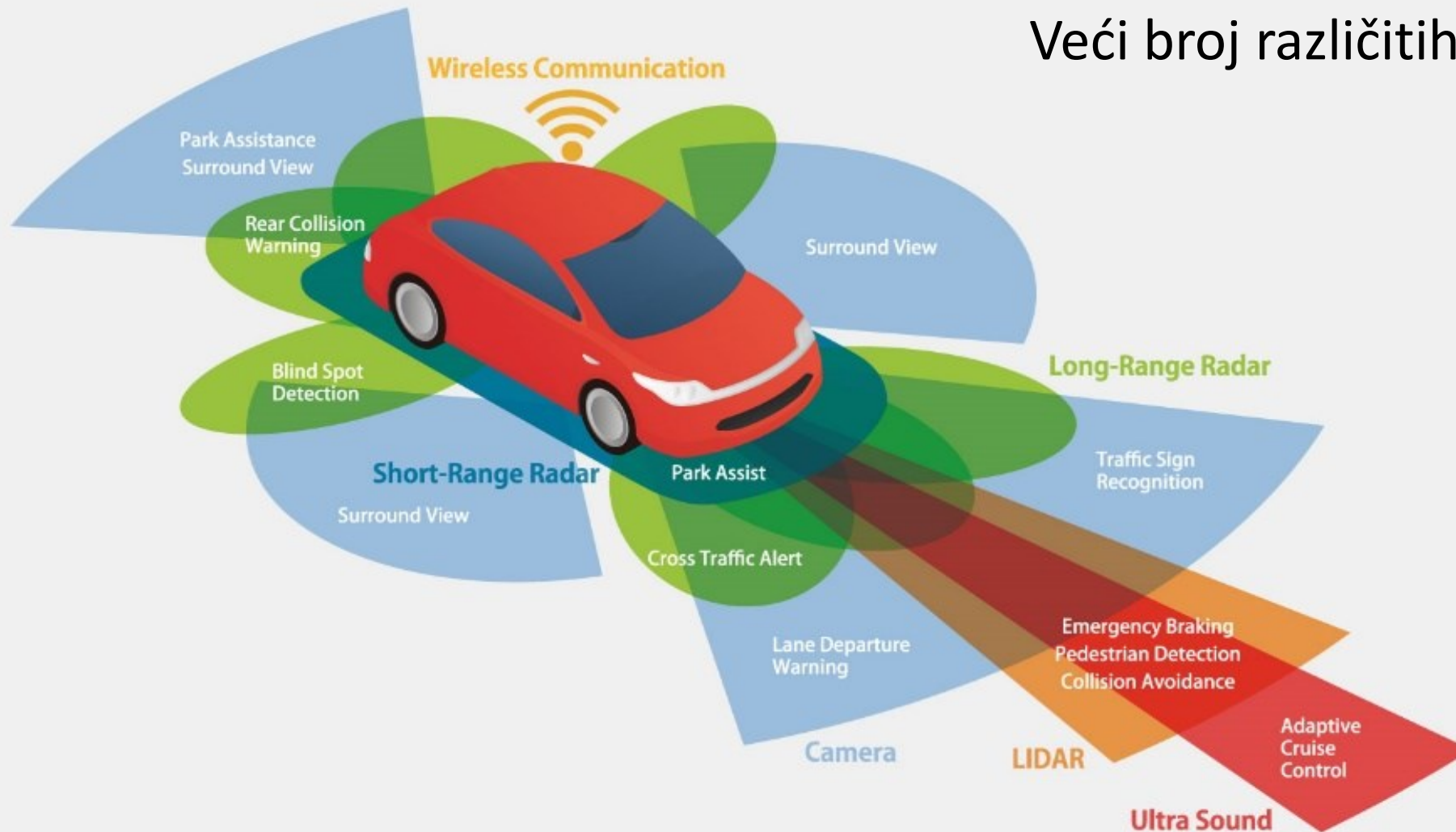
Front, rear, and wing mounted lidar modules aide in the detection of obstacles in close proximity to the vehicle as well as smaller ones that can get lost in blind spots.

Custom designed compute and storage allow for real-time processing of data. A fully integrated cooling solution keeps components running optimally.



Senzori u ADAS

Veći broj različitih senzora - robusnost



Ključne radne karakteristike senzora u ADAS

Gledište performanse	Čovjek	AV			CV	CAV
		<i>Radar</i>	<i>Lidar</i>	<i>Kamera</i>	<i>DSRC</i>	<i>CV+AV</i>
Detekcija objekata	Dobro	Dobro	Dobro	Zadovolj.	-	Dobro
Klasifikacija objekata	Dobro	Loše	Zadovolj.	Dobro	-	Dobro
Procjena udaljenosti	Zadovolj.	Dobro	Dobro	Zadovolj.	Dobro	Dobro
Detekcija rubova	Dobro	Loše	Dobro	Dobro	-	Dobro
Praćenje voznog traka	Dobro	Loše	Loše	Dobro	-	Dobro
Doseg vidljivosti	Dobro	Dobro	Zadovolj.	Zadovolj.	Dobro	Dobro
Performanse po lošem vremenu	Zadovolj.	Dobro	Zadovolj.	Loše	Dobro	Dobro
Performanse pri lošem osvjetljenju	Loše	Dobro	Dobro	Zadovolj.	-	Dobro
Mogućnost komunikacije s ostatkom prometa i infrastrukture	Loše	-	-	-	Dobro	Dobro

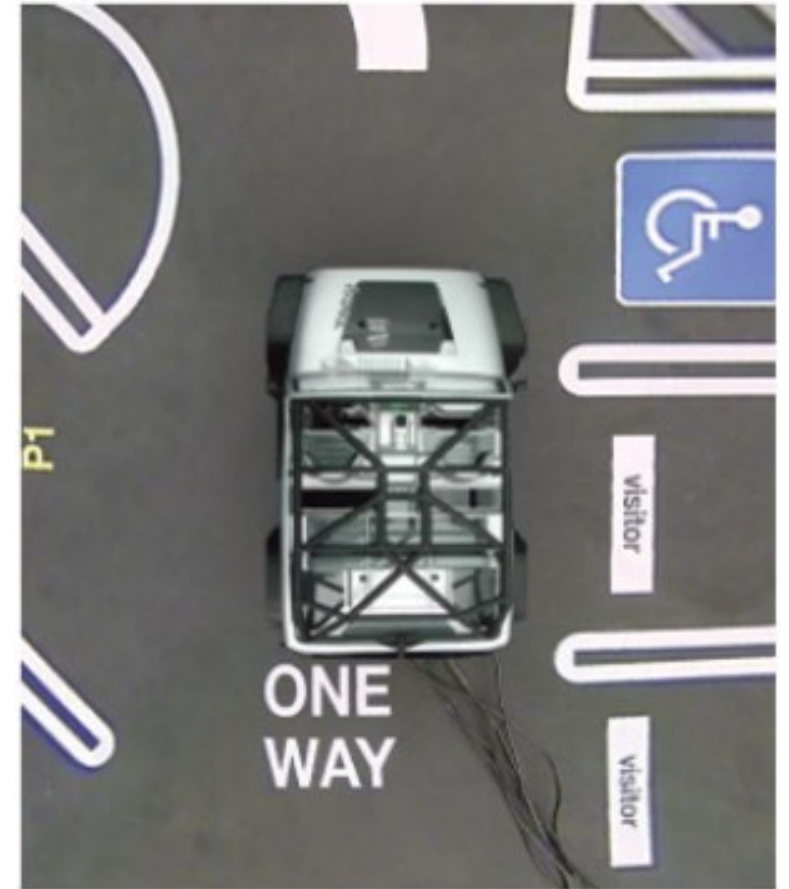
Računalni vid & ADAS – Primjeri...



Računalni vid & ADAS – Primjeri...



*Surround
view*



Računalni vid & ADAS – Primjeri...

*Surround
view*



Računalni vid & ADAS – Primjeri...

1. Original image

frame 414

Confidence:

Line0	Line1	Line2	Line3
94%	100%	100%	0%

Alert! Lane departure!

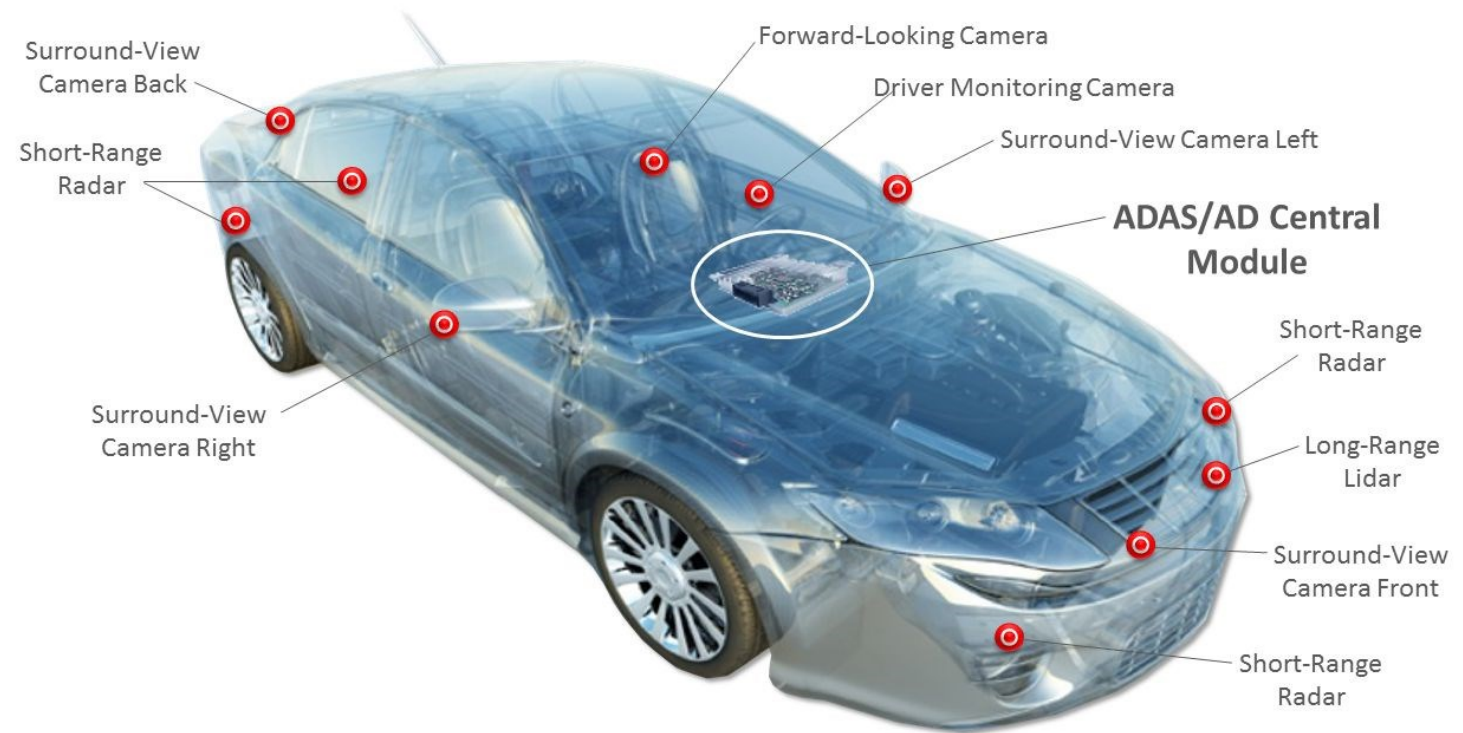


P i t a n j a ?

Kamere za ADAS

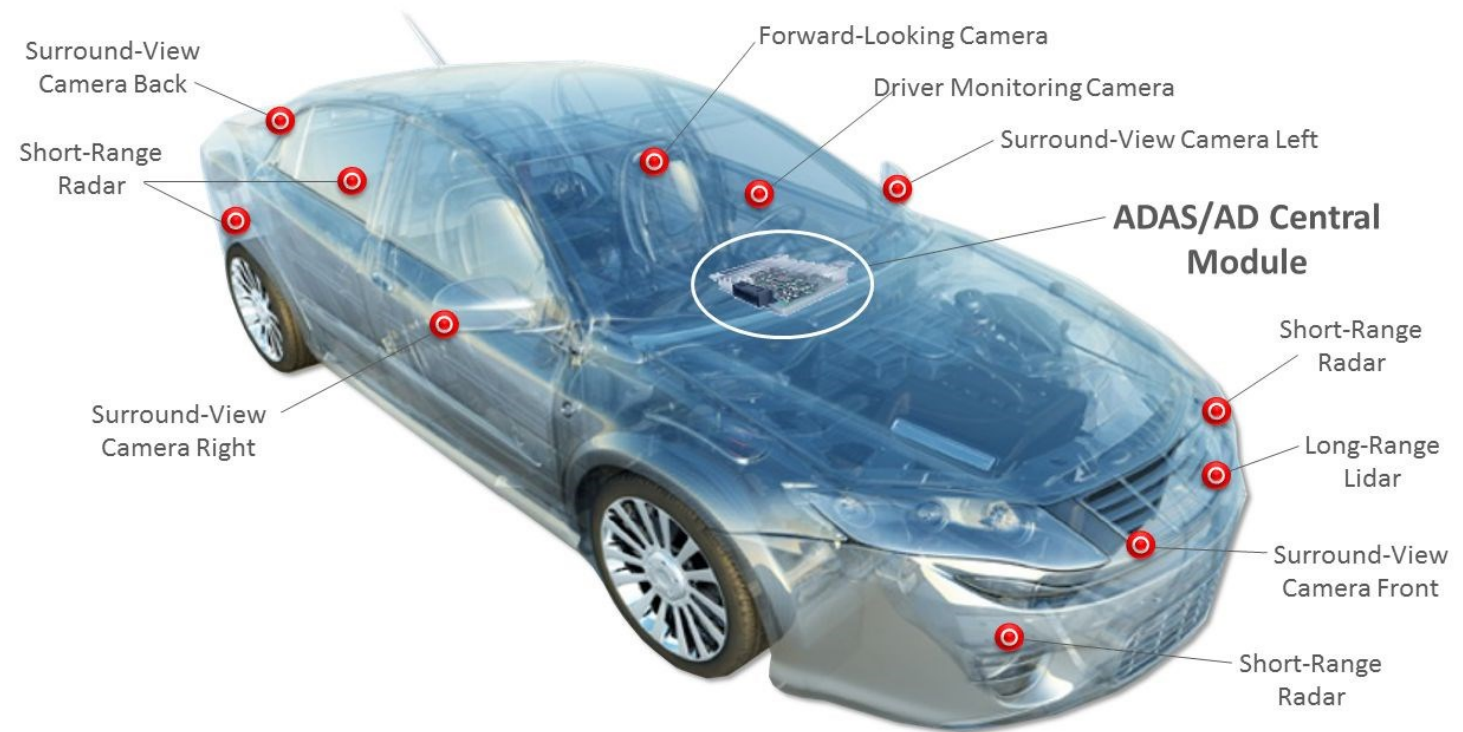
Kamere za ADAS – čemu služe?

- Pružaju informacije vozaču o okolini vozila
 - prednja kamera
 - stražnja kamera
 - kamere u retrovizorima
 - Kamera u upravljačkoj ploči
- Pružaju ulazne podatke algoritmima koji se koriste u autonomnoj vožnji



Kamere za ADAS – centralizirani pristup

- Različiti tipovi kamera, svaka ima odgovarajuće karakteristike
- Današnji automobili često koriste **centralizirani sustav**, gdje jedna centralna jedinica obrađuje sliku s više (npr. 6) kamera
 - Procesiranje se obavlja softverski – procesor se suočava s teškim zahtjevima
 - Velika potrošnja
 - Potreba za velikim kapacitetima za pohranu podataka



Kamere za ADAS – centralizirani pristup

- Centralizirana obrada slike
- Trenutni sustavi digitalnih kamera primaju sirove (engl. *raw*) podatke koji se potom obrađuju i prosljeđuju ekrana za prikaz



- Image Capture
- Image Compression
- Video Streaming

Camera



- Image decoding
- Lens correction
- Geometrical transformation (top-view)
- Video Stream Transcoding
- Video Analytics
- Overlay
- Image streaming

„Camera ECU“



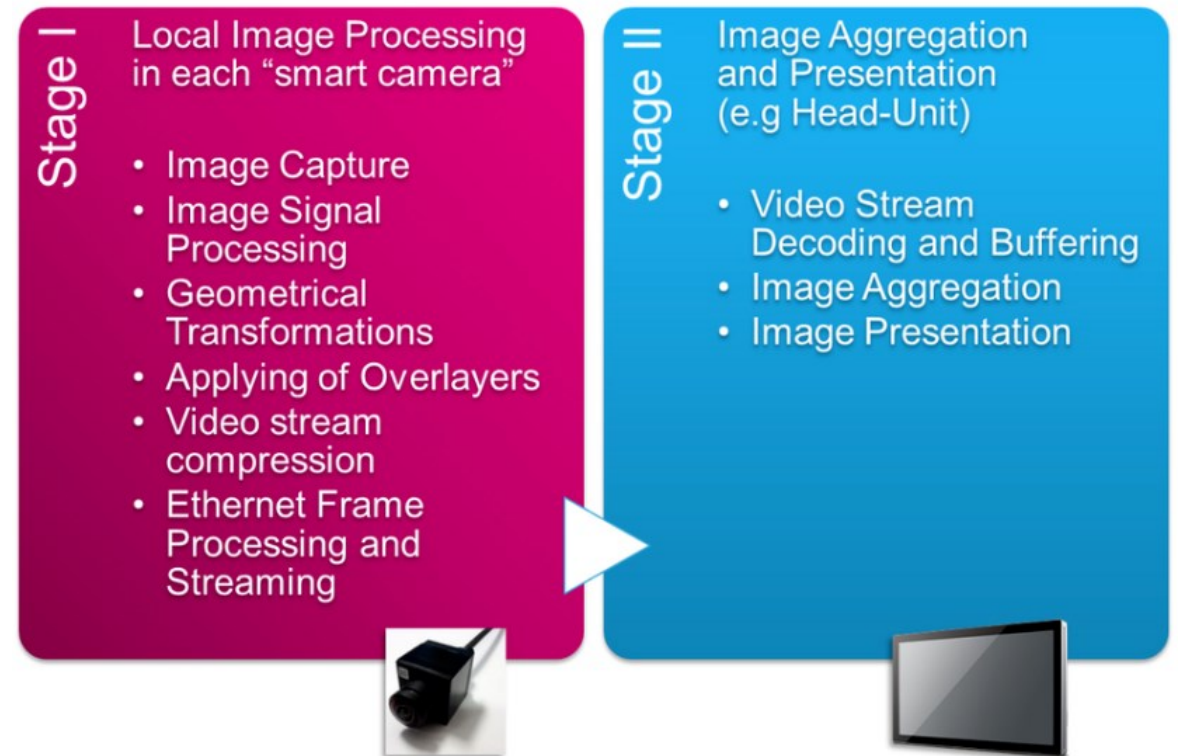
- Image decoding
- Image Aggregation
- Image Display

Head-Unit

Kamere za ADAS – decentralizirani pristup

- **Decentralizirani pristup**

- prebacivanje većina procesiranja slike na samu kameru → automotive smart camera (ugrađeni algoritmi predobrade + dodatna jednostavnija obrada, npr. detekcija rubova)
- Nema potrebe za „Camera ECU”
- Algoritmi više razine tada se na ECU izvode sa smanjenim računalnim zahtjevima
- Individualne slike se pred-obrađuju unutar same kamere i šalju se glavnom procesoru preko komunikacijskog sučelja



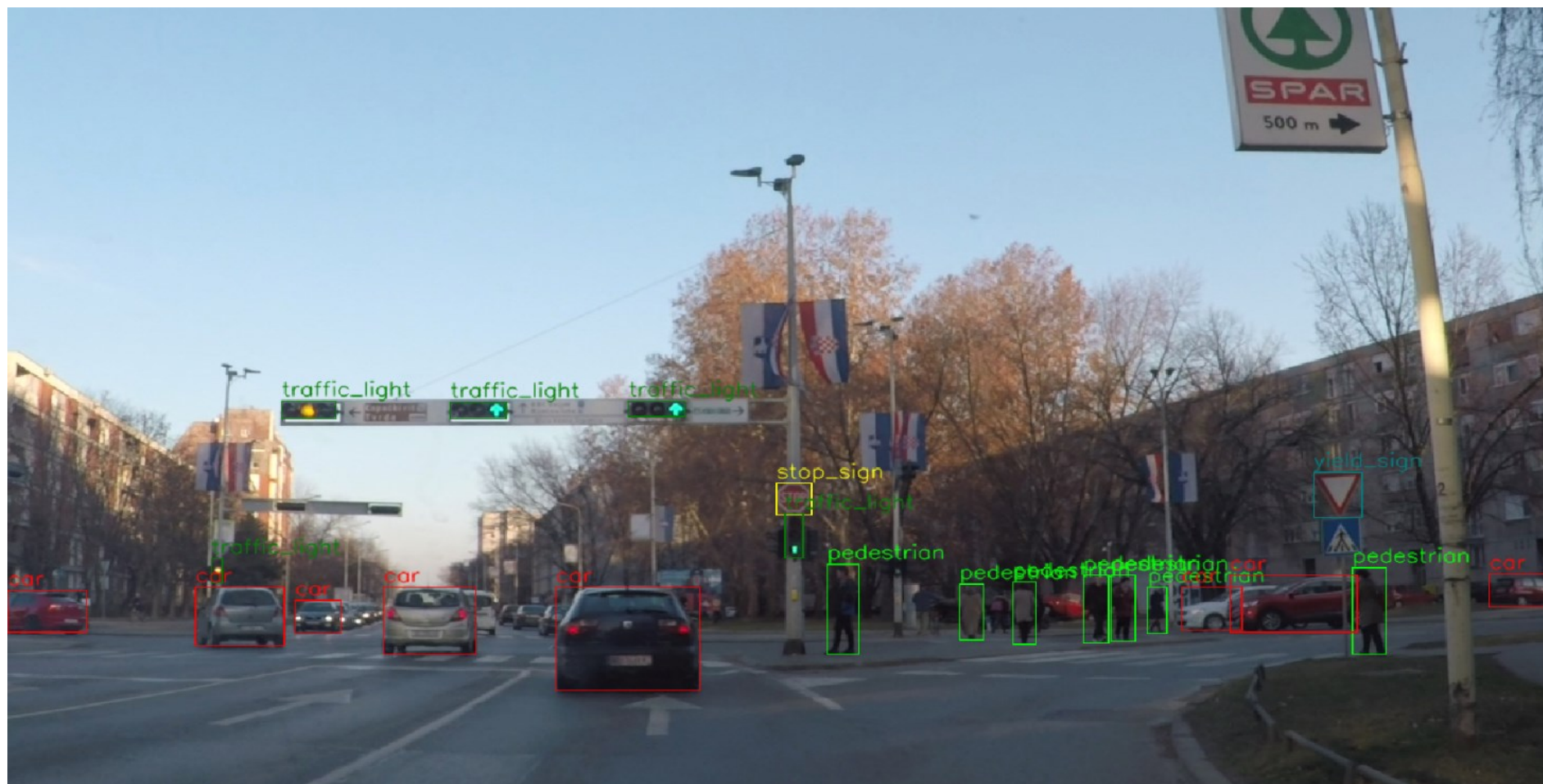
Kamere za ADAS – Primjer *automotive smart* kamere

- 132dB HDR Imager (Vx6640)
 - 1280x960, up to 60fps
- Companion Chip (STV0991)
 - ARM R4 @ 500MHz
 - No external RAM/Flash
 - AutoSAR, FreeRTOS, SafeRTOS*
 - HDR ISP and adv. graphics processor
 - MJPEG, H.264 (Profile Level 4.1)
 - HW-timestamp, 802.1AS-2011, 802.1Q-2011
 - Transport Protocols (incl. HW-timestamp)
 - RTP over UDP (RFC 3550)
 - RTP A/V over UDP (RFC 3551)
 - AVTP (IEEE 1722a)
 - Built-in Video Analytics
 - Optical Flow
 - Edge Detection



Kamere za ADAS – primjer slike s prednje kamere

- **PREDNJA KAMERA** – za srednji i veliki domet (100-250m)



Kamere za ADAS – prednja kamera

- **PREDNJA KAMERA** – za srednji i veliki domet (100-250m)
- Koristi se u algoritmima za automatsku detekciju objekata, klasifikaciju objekata, određivanje udaljenosti do njih
- Npr. mogu identificirati
 - Pješake, bicikliste, motorna vozila,
 - Prometne znakove i signalizaciju,
 - Bočne vozne trake, rubove ceste, stupove mosta, ...
- **Kamere srednjeg dometa** – upozoravaju vozača o nailasku na raskrižje, pješake, kočenje u nuždi zbog objekta ispred, detekcija vozničkih traka,...
- **Kamere velikog dometa** – prepoznavanje prometnih znakova, kontrola udaljenosti,...

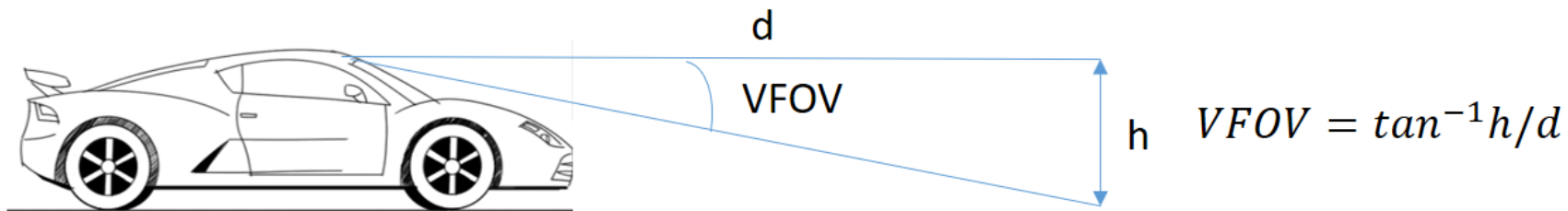
Kamere za ADAS – prednja kamera

- **PREDNJA KAMERA** – za srednji i veliki domet (100-250m)
- Glavna razlika između prednjih kamera za srednji i visoki domet je vidno polje ili FoV (engl. *Field of View*)
 - Za sustave srednjeg dometa koristi se horizontalni FoV od 70 ° do 120 °
 - Za sustave visokog dometa koristi se horizontalni FoV od oko 35 °
- Što je FoV? Pogledajmo...

Parametri kamera za ADAS

Vidno polje (engl. *Field of View* – FoV)

- Dio prostora obuhvaćen kamerom, izražen u stupnjevima (°)
- Horizontalno (HFoV) i vertikalno (VFoV) vidno polje
- HFoV
 - prednja kamera HFoV od 35 ° pa na više (ovisno o aplikaciji)
 - *surround view* kamere obično HFoV 120° do 180°
 - kamere za nadzor vozača HFoV 40° do 50°
- VFoV - ovisi o visini na kojoj je montirana kamera i udaljenosti objekata od interesa



Parametri kamera za ADAS

Rezolucija

- Izražava se u broju piksela (širina x visina) – npr. 1920x1080 (Full HD), 3840x2160 (UHD)
- Često se izražava i u mjernoj jedinici [piksela/°]
- Zahtjevi se razlikuju ovisno o namjeni, npr.
 - Nadzor okoline > 15 piksela/1°
 - Prepoznavanje znakova ili nadzor vozača traži veću rezoluciju
- Današnje kamere za ADAS oko 1-2 Mpixel
- Budući sustavi pokušat će pokrivati srednji i veliki domet isključivo optičkim sustavom
→ da bi to uspjelo, senzori slike u budućnosti trebat će imati minimalno 7 Mpixel

Kamere za autonomnu vožnju i tzv. *consumer* kamere se bitno razlikuju!
(svrha: *vision processing* vs. *viewing by humans*)

Parametri kamera za ADAS

Broj okvira u sekundi (engl. *frames per second*)

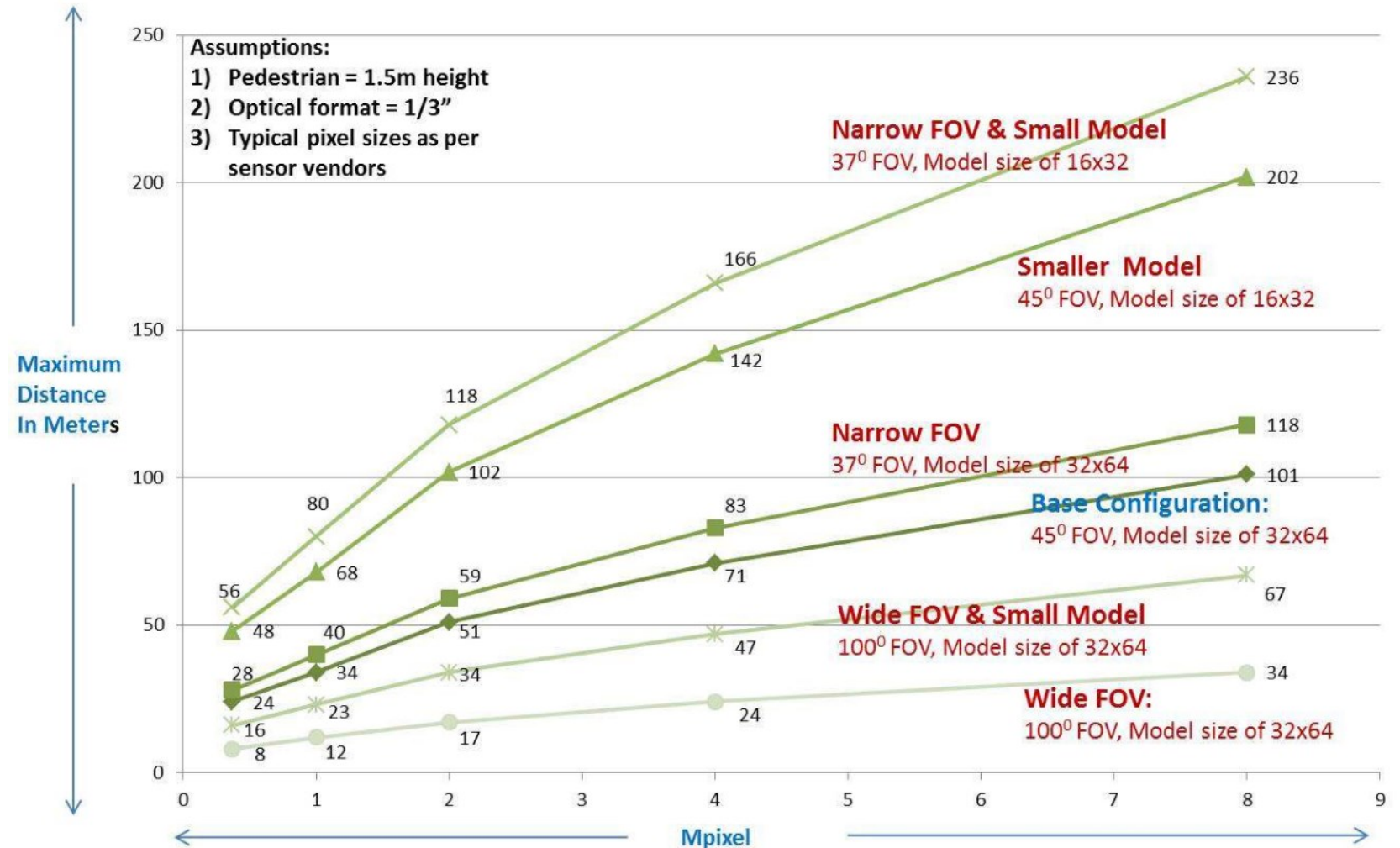
- Broj slika koje se izmjenjuju u jednoj sekundi, kako bi se prikazao kontinuirani pokret

Dinamičko područje (engl. *Dynamic range*)

- Određeno je razlikom između najtamnije i najsvjetlije razine koja se može prikazati
- Najniža (najtamnija) razina je određena razinom šuma, a najviša (najsvjetlija) razina je određena razinom zasićenja slikovnog senzora kamere
- Razina svjetline u okolini u kojoj se odvija promet može biti vrlo raznolika i dinamično se mijenja (ulazak u tunele ili garaže, sunčano/oblačno vrijeme)
- Pravilnim dizajnom može se postići dinamičko područje kamere do 145 dB (tzv. *High Dynamic Range - HDR sensors*)

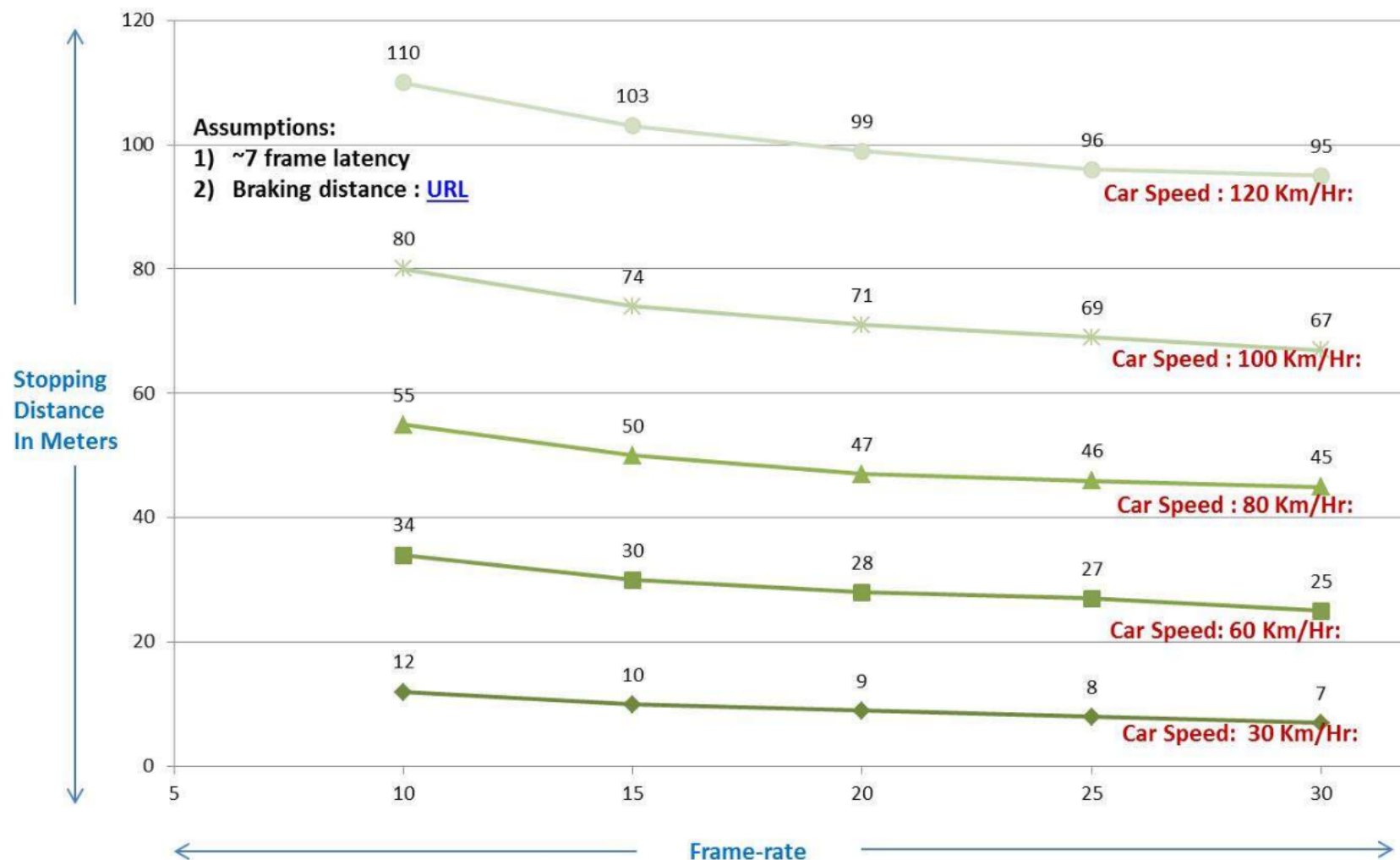
Kamere za ADAS – utjecaj rezolucije !

- *Mpixel vs. Maksimalna udaljenost za detekciju pješaka*
- Što se dobiva s više Mpixel-a?
- Što se dobiva s većim FoV?
- Što se događa sa računalnim zahtjevima?



Kamere za ADAS – utjecaj broja *fps* !

- **Broj fps vs. Dužina zaustavnog puta**
- Primjer – automatsko kočenje prilikom detekcije objekta ispred vozila
- Stopping distance – udaljenost od trenutka detekcije objekta do potpunog zaustavljanja vozila
- Praćenje objekta kroz nekoliko okvira (povećanje efikasnosti detekcije) + samo kočenje vozila



P i t a n j a ?

Postupci za predobradu slike



Sadržaj

- Prostorno filtriranje slike
 - Tipovi filtera
 - Primjena u ADAS algoritmima (autonomna vožnja)
- Uklanjanje šuma iz slike s kamere
- Histogram slike i primjena histograma

Prostorno filtriranje i uklanjanje šuma iz slike

Postupci za predobradu digitalne slike

- Rade se za potrebe poboljšanja kvalitete slike
- Mogu se provoditi u
 - **Prostornoj domeni**
 - Frekvencijskoj domeni (domeni prostornih frekvencija)
- **Prostorna domena**
 - Odnosi se na samu ravninu (plohu) slike
 - Na sliku o kakvoj smo do sada pričali (2D polje)
 - Metode obrade slike zasnovane na izravnoj manipulaciji na pikselima

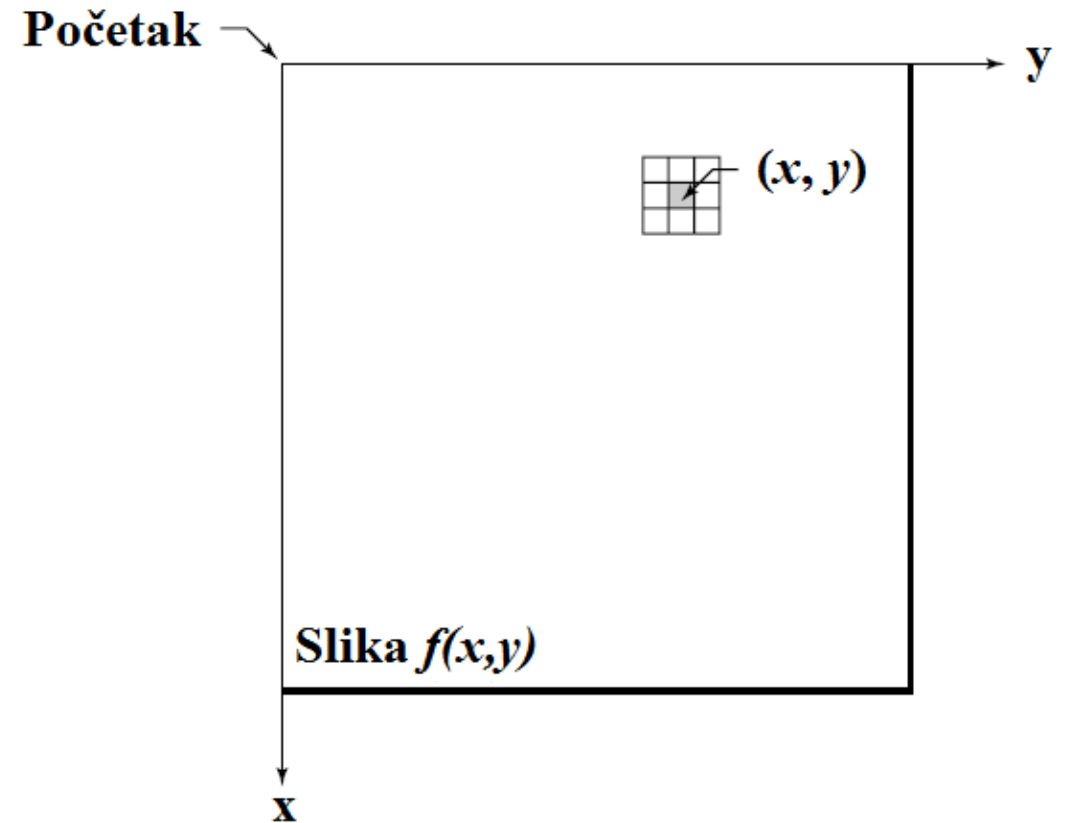
Postupci za predobradu digitalne slike

- Dvije najvažnije kategorije obrade slike u prostornoj domeni
 - **Promjene intenziteta** (na skali sivog, eng. *gray-level*)
 - Na pojedinačnom pikselu
 - Npr. promjena svjetline, promjena kontrasta
 - **Prostorno filtriranje**
 - Na grupi piksela
 - Npr. niskopropusno, visokopopusno filtriranje

$$g(x,y) = T [f(x,y)]$$

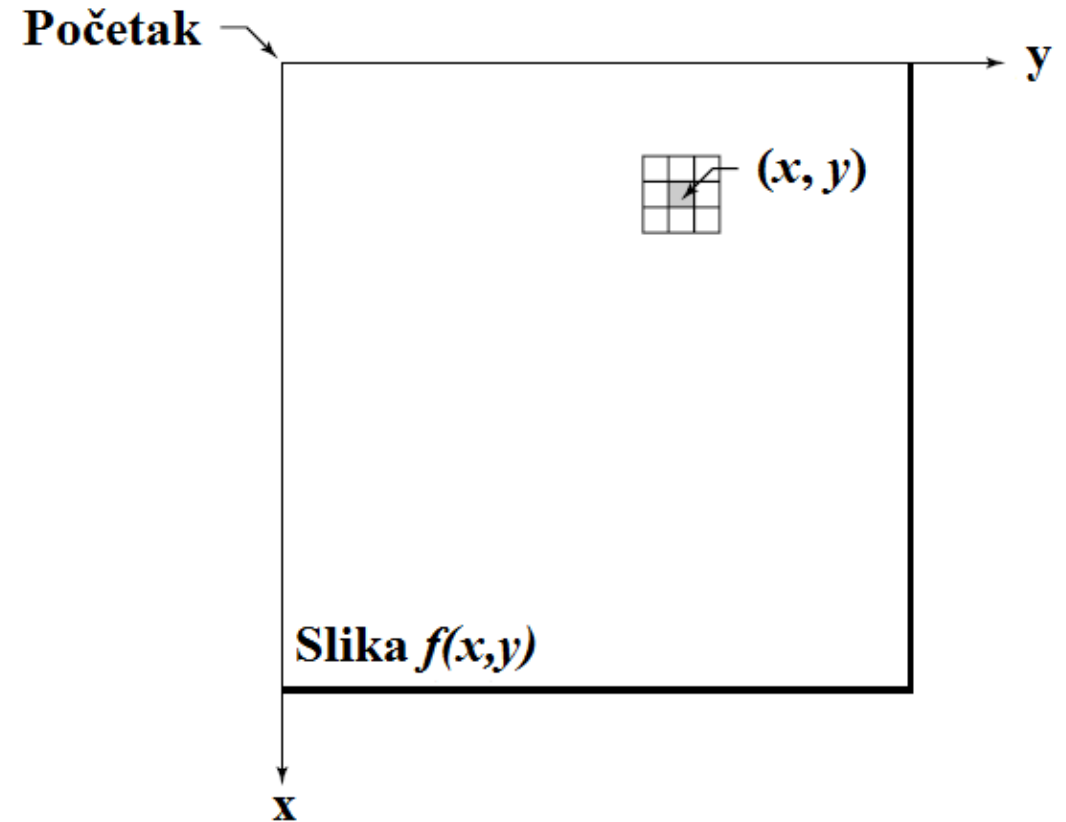
Prostorno filtriranje slike

- Filtriranje u prostornoj domeni
 - koristi se **konvolucijska maska (matrica)**, koja se primjenjuje na grupu piksela - **konvolucijski kernel**
 - 1D prostorno filtriranje (konvolucijska maska ja vektor)
 - **2D prostorno filtriranje** (konvolucijska maska ja matrica)



Prostorno filtriranje slike

- Osnovni pristup – definiranje susjedstva u prostoru
 - Obično kvadratno područje sa središtem u točki (x,y)
 - Središte se pomiče od piksela do piksela, počevši npr. od gornjeg lijevog ugla
 - Maska se primjenjuje na svakoj lokaciji (x,y) kako bi se dobila slika g
- Množenje svakog piksela u okolini s odgovarajućim koeficijentom i sumiranje rezultata kako bi se dobio rezultat filtriranja za svaku točku (x,y)
- Koeficijenti su grupirani u matricu, nazvanu maska filtra (konvolucijska maska, konvolucijski kernel)



$$g(x,y) = T [f(x,y)]$$

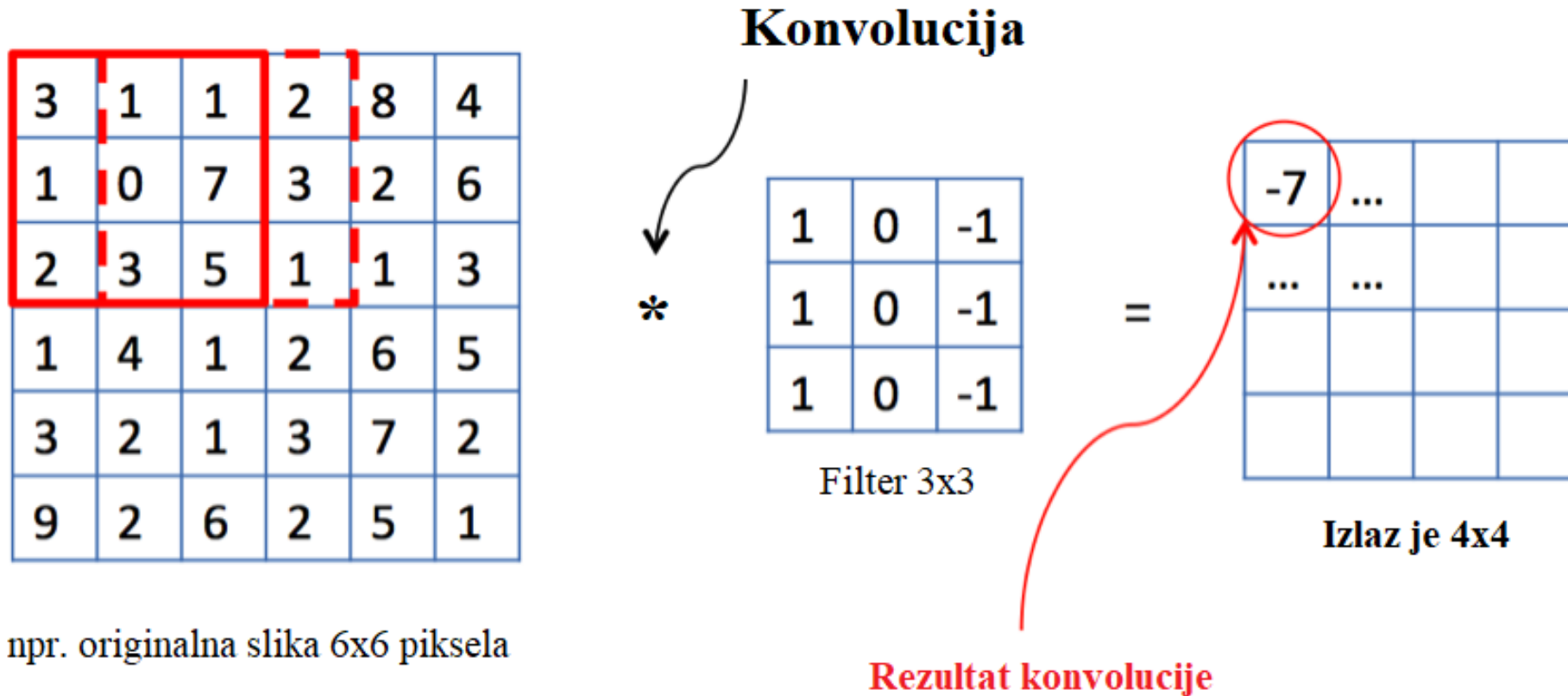
Prostorno filtriranje slike – princip

a	b	c
d	e	f
g	h	i

P _{0,0}	P _{1,0}	P _{2,0}	P _{3,0}	P _{4,0}					
P _{0,1}	P _{1,1}	P _{2,1}	P _{3,1}	P _{4,1}					
P _{0,2}	P _{1,2}	P _{2,2}	P _{3,2}	P _{4,2}					
P _{0,3}	P _{1,3}	P _{2,3}	P _{3,3}	P _{4,3}					

$$P_{2,1}' = a * P_{1,0} + b * P_{2,0} + c * P_{3,0} + d * P_{1,1} + e * P_{2,1} + f * P_{3,1} + g * P_{1,2} + h * P_{2,2} + i * P_{3,2}$$

Što s filtriranjem piksela na rubovima slike...?



- Dio maske je „izvan” područja slike – ako ne definiramo što se u tom dijelu događa, filtrirana slika je manje rezolucije od originalne

Što s filtriranjem piksela na rubovima slike...?

- Dopunjavanje slike nulama, do potrebnih dimenzija (ovise o veličini, tj. dimenzijama maske filtra) → tzv. *zero padding* → kako bi sačuvali dimenzije ulazne slike

0	0	0	0	0	0	0	0
0	3	1	1	2	8	4	0
0	1	0	7	3	2	6	0
0	2	3	5	1	1	3	0
0	1	4	1	2	6	5	0
0	3	2	1	3	7	2	0
0	9	2	6	2	5	1	0
0	0	0	0	0	0	0	0

npr. originalna slika 6x6 piksela

Konvolucija



1	0	-1
1	0	-1
1	0	-1

Filter 3x3

=

Izlaz je 6x6

(dimenzije kao i originalna slika)

- Postoje i drugi tipovi dopunjavanja slike, npr. dopunjavanje preslikavanje (engl. *mirroring*)

Količina detalja u slici...jesu li svi potrebni?



Prostorno filtriranje slike

- Što znači niskopropusno filtriranje?
- Što znači visokopropusno filtriranje?
- Što bi to bilo nisko/visoko što se propušta u slici?



Niskopropusno filtriranje

- Niskopropusno filtriranje – guši visoke prostorne frekvencije (detalji i rubovi u slici), propušta samo niske prostorne frekvencije
- **Omekšava oštre rubove, uklanja dio detalja iz slike (zamuti ih)**
- Primjeri:
 - ***Moving average*** → primjer maske za 3x3 filter
 - **Gaussov filter** → veću težinu daje pikselima u središtu kernela

$$\begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Niskopropusno filtriranje

- **Medijan filter**

- Posebna vrsta filtriranja, gdje se kao rezultat uzima medijan vrijednosti svih piksela u $N \times N$ okolini piksela koji se trenutno obrađuje
- **Učinkovito uklanja „salt and pepper” šum**

- **Bilateralni filter**

- Zamjenjuje vrijednost piksela srednjom vrijednošću okolnih elemenata pomnoženih težinama koje ovise o udaljenosti od centralnog elementa slike, ali ovise i o Gaussovoj distribuciji
- **Čuva rubove, a uklanja šum u određenoj mjeri**

Niskopropusno filtriranje

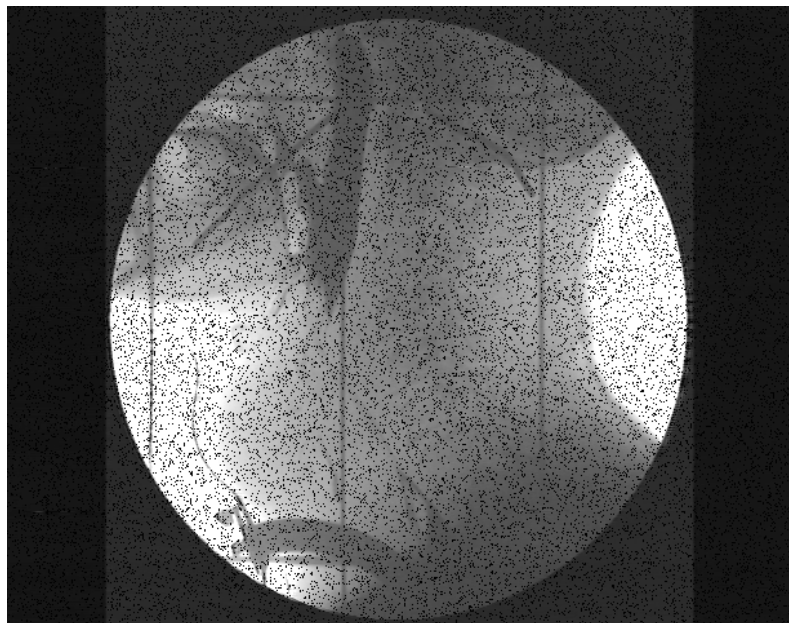
- ***Non-local Means Denoising*** – brzi postupak koji čuva rubove i efikasno čisti šum
- Open CV
 - `cv2.fastNlMeansDenoisingColored(image, None, h, hC, TwS, Sws)`
 - Sws = 21 za manju snagu šuma, a 35 za veće snage šuma
 - TwS = 3, 5, 7, 9 ili 11 (uzima se veći što je šum veće snage)
 - h – snaga filtriranja za luminantnu komponentu, veći h znači jače filtriranje, lošije očuvani rubovi, uzima se između 5 i 10
 - hC – snaga filtriranja za komponente boje

Uklanjanje šuma iz slike NP filtriranjem

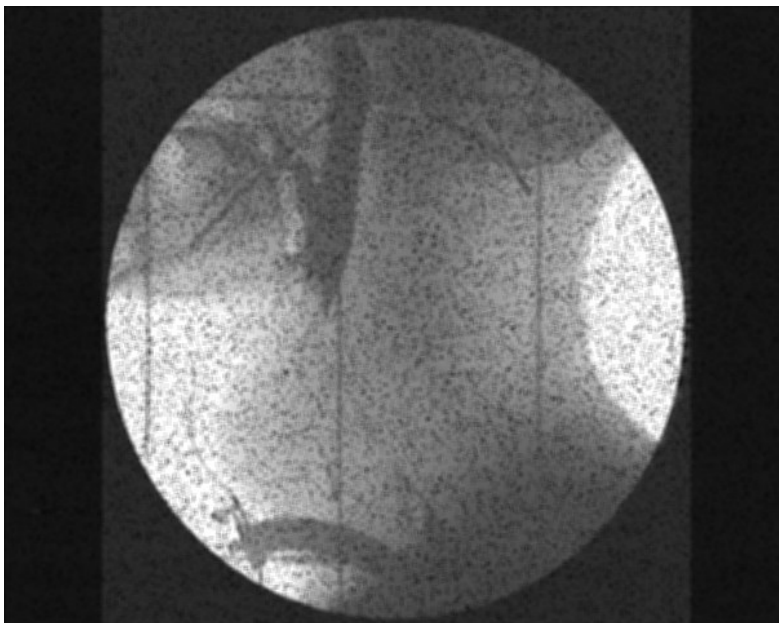
- Šum se u slici pojavljuje najčešće kod loših uvjeta snimanja (niska razina osvjetljenosti, magla, kiša...)
- Gaussovim ili Medijan filtrom može se do neke mjere smanjiti šum, **ali se ovakvim filtriranjem ne čuvaju rubovi**
- Bilateralni filter **čuva rubove i do neke mjere čisti šum**
- *Non-local Means Denoising* – **brzi postupak koji čuva rubove i efikasno čisti šum**

Uklanjanje šuma iz slike NP filtriranjem

- Smanjivanje „*salt and paper*” šuma u slici → Medijan filter



original + šum



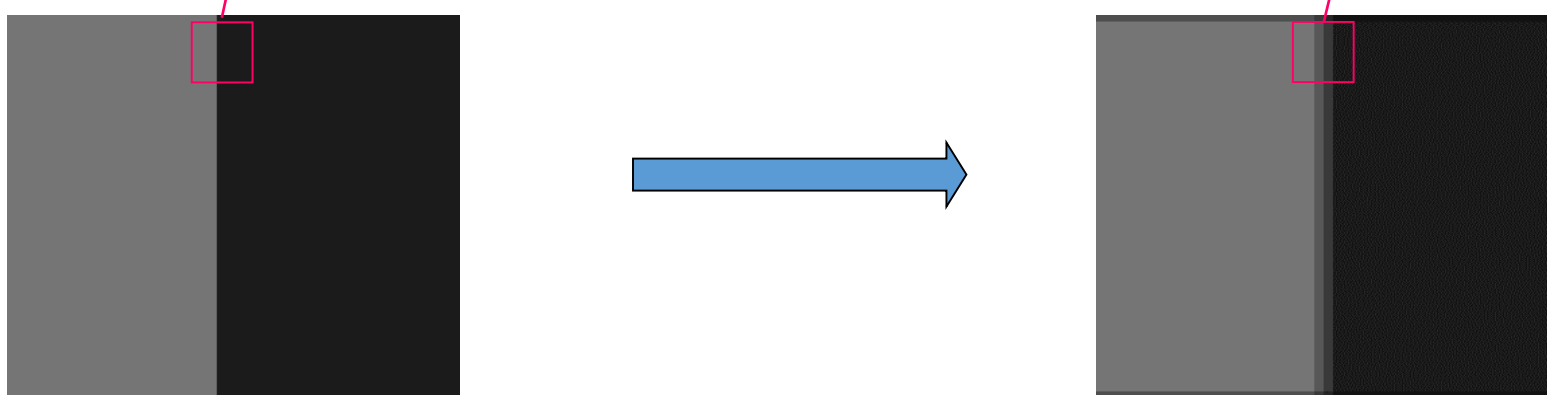
Gaussov filter



Medijan filter

Prostorno filtriranje – *moving average* filter

- Niskopropusno filtriranje
 - Omekšava oštre rubove, zamučuje detalje
- Primjeri:
 - *Moving average* → primjer maske za 3x3 filter

$$\begin{bmatrix} 117 & 117 & 27 & 27 \\ 117 & 117 & 27 & 27 \\ 117 & 117 & 27 & 27 \\ 117 & 117 & 27 & 27 \end{bmatrix} * \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix} = \begin{bmatrix} 117 & 105 & 57 & 27 \\ 117 & 105 & 57 & 27 \\ 117 & 105 & 57 & 27 \\ 117 & 105 & 57 & 27 \end{bmatrix}$$


Prostorno filtriranje – *moving average* filter



Moving average 11 x 11 filter

Prostorno filtriranje - Gaussov filter



Gaussov filter 9 x 9

Prostorno filtriranje - Medijan filter



Medijan filter 9 x 9

Prostorno filtriranje - Bilateralni filter



Bilateralni filter veličine 15

Filtriranje slike u OpenCV

```
import cv2
import numpy as np

image = cv2.imread('images/car_and_road.jpg')
cv2.imshow('Original Image', image)
cv2.waitKey(0)

# Creating our 3 x 3 kernel
kernel_3x3 = np.ones((3, 3), np.float32) / 9

# We use the cv2.filter2D to convolve the kernel with an image
blurred = cv2.filter2D(image, -1, kernel_3x3)
cv2.imshow('3x3 Kernel Blurring', blurred)
cv2.waitKey(0)

# Creating our 7 x 7 kernel
kernel_7x7 = np.ones((7, 7), np.float32) / 49

blurred2 = cv2.filter2D(image, -1, kernel_7x7)
cv2.imshow('7x7 Kernel Blurring', blurred2)
cv2.waitKey(0)

cv2.destroyAllWindows()
```

```
import cv2
import numpy as np

image = cv2.imread('images/car_and_road.jpg')

# Averaging done by convolving the image with a normalized box filter.
# This takes the pixels under the box and replaces the central element
# Box size needs to be odd and positive
blur = cv2.blur(image, (3,3))
cv2.imshow('Averaging', blur)
cv2.waitKey(0)

# Instead of box filter, gaussian kernel
Gaussian = cv2.GaussianBlur(image, (7,7), 0)
cv2.imshow('Gaussian Blurring', Gaussian)
cv2.waitKey(0)

# Takes median of all the pixels under kernel area and central
# element is replaced with this median value
median = cv2.medianBlur(image, 5)
cv2.imshow('Median Blurring', median)
cv2.waitKey(0)

# Bilateral is very effective in noise removal while keeping edges sharp
bilateral = cv2.bilateralFilter(image, 9, 75, 75)
cv2.imshow('Bilateral Blurring', bilateral)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Non-local means denoising u OpenCV

- `cv2.fastNlMeansDenoisingColored(image, None, h, hC, TwS, Sws)`
 - Sws = 21 za manju snagu šuma, a 35 za veće snage šuma
 - TwS = 3, 5, 7, 9 ili 11 (uzima se veći što je šum veće snage)
 - h – snaga filtriranja za luminantnu komponentu, veći h znači jače filtriranje, lošije očuvani rubovi, uzima se između 5 i 10
 - hC – snaga filtriranja za komponente boje

```
import numpy as np
import cv2

image = cv2.imread('images/car_and_road.jpg')

# Parameters, after None are - the filter strength 'h' (5-10 is a good range)
# Next is hForColorComponents, set as same value as h again
#
dst = cv2.fastNlMeansDenoisingColored(image, None, 6, 6, 7, 21)

cv2.imshow('Fast Means Denoising', dst)
cv2.waitKey(0)

cv2.destroyAllWindows()
```


Non-local means denoising rezultati...



Visokopropusno filtriranje

- Koristi se za izoštravanje slike → primjer maske filtra (konvolucijske matrice)

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$



Filtar 3 x 3

P i t a n j a ?

Histogram slike i primjena histograma

Što je histogram slike?

- Histogram slike s L mogućih razina intenziteta u području $[0, L-1]$ je diskretna funkcija

$$h(r_k) = n_k$$

- r_k je k-ta razina intenziteta, a n_k je broj piksela koji imaju tu razinu intenziteta (svjetline ili boje)

- Često se koristi normalizirani histogram

$$p_r(r_k) = \frac{h(r_k)}{\sum n_k} = \frac{n_k}{n}$$

gdje je n ukupan broj piksela

Histogram slike

- Idealan je za vizualizaciju pojedinih komponenata boje u slici
- Funkcija za proračun histograma u OpenCV ...



`cv2.calcHist(images, channels, mask, histSize, ranges)`

- `images` : it is the source image of type `uint8` or `float32`. it should be given in square brackets, ie, "[img]".
- `channels` : it is also given in square brackets. It is the index of channel for which we calculate histogram. For example, if input is grayscale image, its value is [0]. For color image, you can pass [0], [1] or [2] to calculate histogram of blue, green or red channel respectively.
- `mask` : mask image. To find histogram of full image, it is given as "None". But if you want to find histogram of particular region of image, you have to create a mask image for that and give it as mask.
- `histSize` : this represents our BIN count. Need to be given in square brackets. For full scale, we pass [256].
- `ranges` : this is our RANGE. Normally, it is [0,256].

Histogram slike u OpenCV

```
import cv2
import numpy as np

# We need to import matplotlib to create our histogram plots
from matplotlib import pyplot as plt

image = cv2.imread('images/car_and_road.jpg')

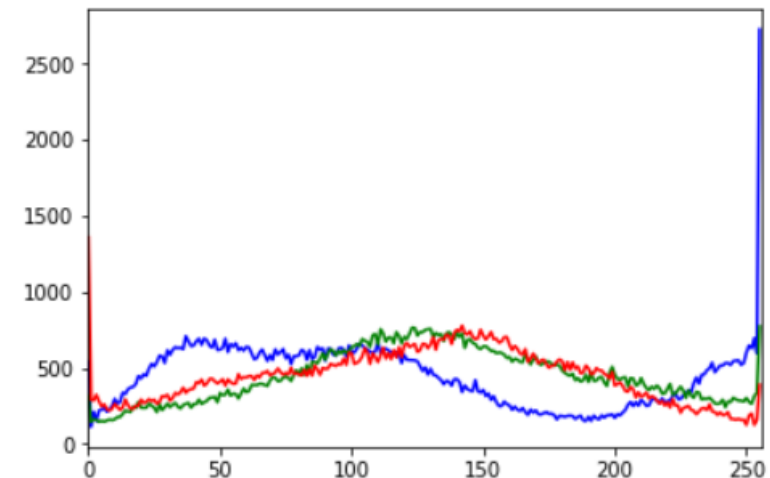
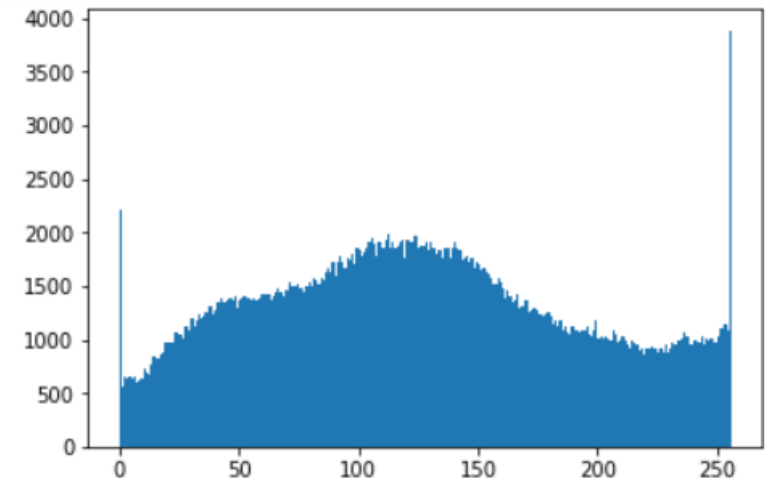
histogram = cv2.calcHist([image], [0], None, [256], [0, 256])

# We plot a histogram, ravel() flatens our image array
plt.hist(image.ravel(), 256, [0, 256]); plt.show()

# Viewing Separate Color Channels
color = ('b', 'g', 'r')

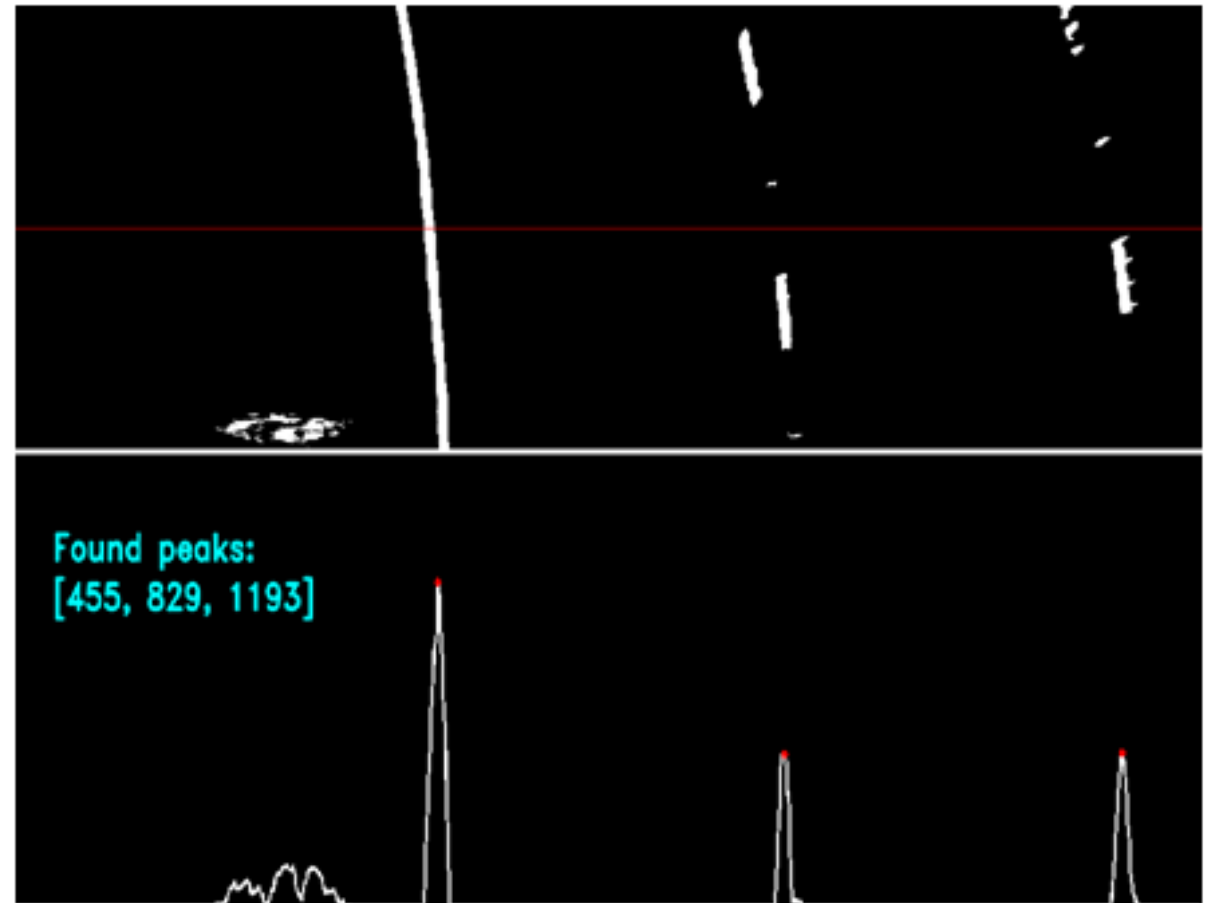
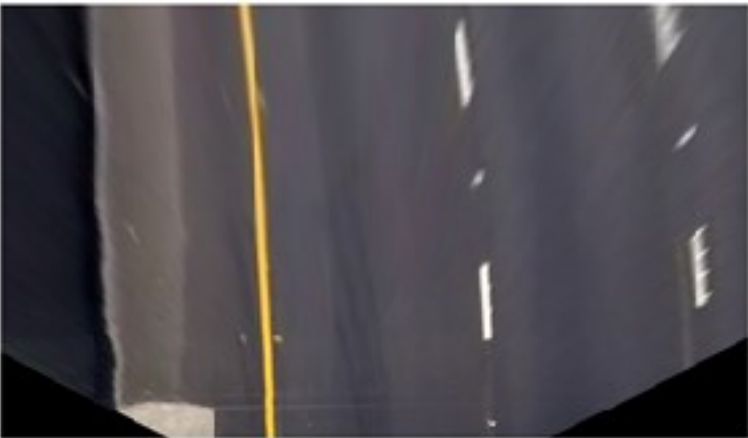
# We now separate the colors and plot each in the Histogram
for i, col in enumerate(color):
    histogram2 = cv2.calcHist([image], [i], None, [256], [0, 256])
    plt.plot(histogram2, color = col)
    plt.xlim([0,256])

plt.show()
```



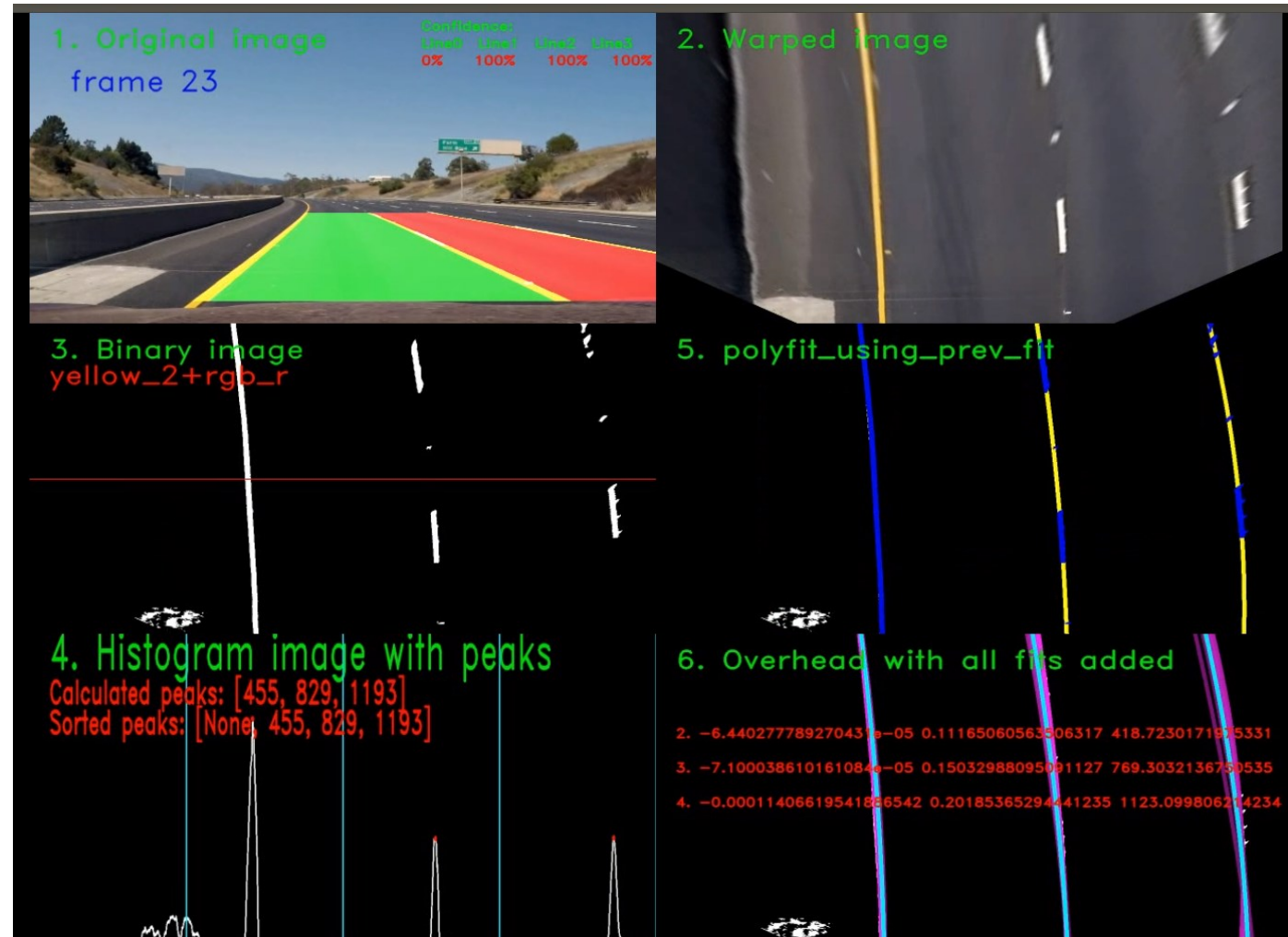
Histogram slike ...drugačiji način primjene

- Detekcija linija vozne trake



Histogram slike ...drugačiji način primjene

- Detekcija linija vozne trake



P i t a n j a ?